ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ

УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

МОСКОВСКИЙ ГОРОДСКОЙ УНИВЕРСИТЕТ УПРАВЛЕНИЯ ПРАВИТЕЛЬСТВА МОСКВЫ ИМЕНИ Ю.М. ЛУЖКОВА

Кафедра экономики городского хозяйства и жилищного права

Направление: 38.03.04 «Государственное и муниципальное управление»

Профиль «Управление цифровыми проектами города»

Форма обучения: очная

Курсовая работа

по учебной дисциплине

«Система управления базами данных»

на тему: «Проектирование и создание базы данных при организации парковочных пространств города Москвы»

Группа/ курс	22ГМУ-УЦП11.2 / 3 курс	
Студент	_____ (подпись)	Берестеньков Л.В.
Подпись руководителя о допуске к защите	_____ (подпись)	канд. техн. наук, Каманин И.О.
Дата защиты	«___»_____ 2025 года	
Оценка	_____	

Москва 2025

Содержание

ВВЕДЕНИЕ 4

РАЗДЕЛ 1. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ. ПОСТРОЕНИЕ КОНЦЕПТАУЛЬНОЙ МОДЕЛИ БАЗЫ ДАННЫХ 6

1.1. Московский паркинг 6

1.2. Построение концептуальной модели в нотации Питера Чена 13

Выводы по первой главе 18

РАЗДЕЛ 2. НОРМАЛИЗАЦИЯ и ПОСТРОЕНИЕ ДАТАЛОГИЧЕСКОЙ МОДЕЛИ 19

2.1. Нормализация. Построение даталогической модели 19

2.2 Добавление справочных таблиц 28

Выводы по второй главе 29

РАЗДЕЛ 3. ПОСТРОЕНИЕ ФИЗИЧЕСКОЙ МОДЕЛИ БАЗЫ ДАННЫХ 30

3.1. Создание таблиц базы данных 30

3.2. Построение типовых запросов на выборку данных 41

Выводы по 3 главе 49

ЗАКЛЮЧЕНИЕ 51

СПИСОК ЛИТЕРАТУРЫ 52

приложение А 55

Содержание справочных таблиц 55

приложение Б 57

Создание экранной формы для таблицы User 57

Приложение В 59

Создание экранной формы для таблицы Parking_space 59

Приложение Г 61
Создание экранной формы для таблицы Transport 61
Приложение Д 64
Создание экранной формы для таблицы Parking 64
Приложение Е 66
Создание экранной формы для таблицы Payment 66

ВВЕДЕНИЕ

Актуальность темы исследования обусловлена растущей потребностью в эффективной организации парковочных пространств в Москве. Увеличение числа автомобилей и ограниченное количество парковочных мест создают значительные проблемы для горожан и властей. Учитывая вышеизложенное, тема работы: «Проектирование и создание базы данных при организации парковочных пространств города Москвы» является значимой и актуальной.

Объектом исследования является предоставление государственной услуги по организации парковочных пространств.

Предметом исследования выступает база данных по учету предоставления государственной услуги по организации парковочных пространств в городе Москве.

Цель исследования: на основе анализа предметной области разработать базу данных по учету предоставления государственной услуги организации парковочных пространств в городе Москве.

Поставленная цель определила необходимость решения следующих **задач**:

1. Провести анализ существующих систем учета и управления парковочными пространствами.
2. Построить концептуальную даталогическую модель базы данных по учету предоставления государственной услуги организации парковочных пространств в городе Москве.
3. Разработать физическую модель базы данных, обеспечивающую эффективное хранение и обработку информации о парковочных местах, пользователях и их транспортных средствах.

Структура работы. Цели и задачи настоящего исследования определили структуру работы. Она состоит из введения, трех глав,

заключения, а также списка использованных источников и литературы, приложений: А, Б, В, Г, Д, Е.

Первая глава посвящена анализу предметной области и построению концептуальной модели системы учета парковочных пространств.

Вторая глава содержит результаты нормализации отношений и построению даталогической модели базы данных.

В третьей главе представлена физическая модель базы данных по учету предоставления государственной услуги организации парковочных пространств в городе Москве, а также продемонстрирован пример работы с базой данных.

РАЗДЕЛ 1. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ. ПОСТРОЕНИЕ КОНЦЕПТАУЛЬНОЙ МОДЕЛИ БАЗЫ ДАННЫХ

1.1. Московский паркинг

Московский паркинг — это система управления парковочными местами в Москве, созданная для оптимизации использования парковочного пространства и повышения удобства для автомобилистов. Проект был запущен 05 октября 2012 года и с тех пор значительно изменил подход к организации парковки в столице [1]. На сегодняшний день в системе зарегистрировано более 360 тысяч парковочных мест, включая как платные, так и бесплатные зоны [2]. В рамках этой системы действуют около 124 тысяч платных парковок, что позволяет поддерживать баланс между спросом и предложением на парковочные места в городе. Тарифы на платную парковку устанавливаются таким образом, чтобы обеспечивать наличие 10-15% свободных мест, что способствует улучшению транспортной ситуации в Москве. [3]

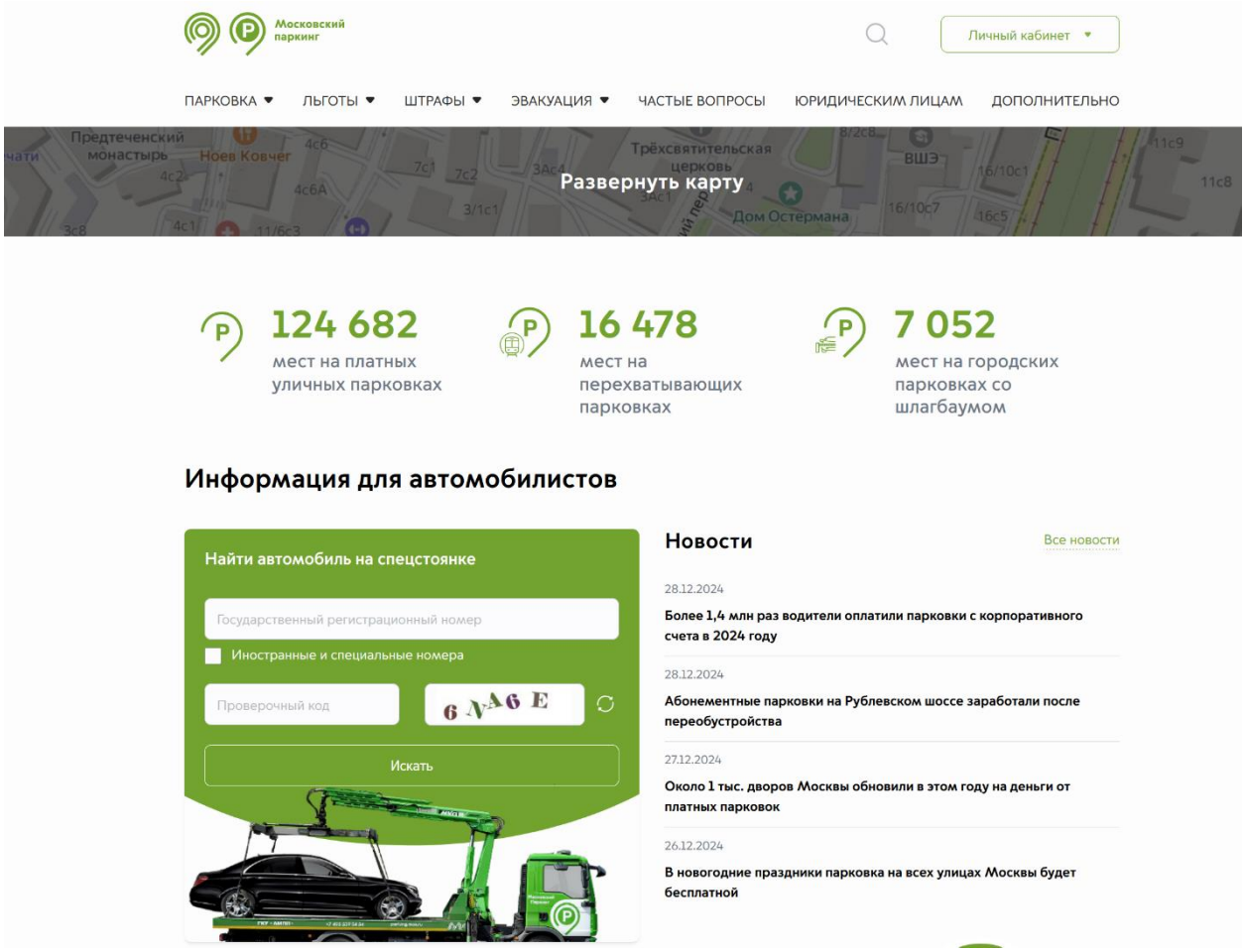


Рисунок 1 Главная страница московского паркинга

Карта парковочных мест Московского паркинга предоставляет пользователям удобный способ поиска и навигации по доступным парковочным местам в Москве. На этой карте отображаются как платные, так и бесплатные парковки, позволяя автомобилистам легко находить подходящие места для стоянки. На карте можно увидеть обозначения различных типов парковок: участки дорог, где разрешено парковаться, а также зоны, где это запрещено. Платные парковки обозначены отдельно, с указанием фиксированной или нефиксированной стоимости [4]. Границы зон парковок выделены синим цветом, что помогает быстро ориентироваться в доступных местах. Пользователи могут взаимодействовать с картой через мобильное приложение «Парковки» или на сайте Московского паркинга [5]. С помощью приложения можно не только найти ближайшую парковку, но и получить

подробную информацию о ней, включая стоимость, количество свободных мест и график работы. Также доступна функция построения маршрута до выбранной парковки. Для удобства пользователей предусмотрена возможность оплаты парковки через приложение, что позволяет избежать очередей у паркоматов. Однако стоит отметить, что оплата доступна только для открытых парковок без шлагбаумов. В случае необходимости пользователи могут изменить данные о своей оплаченной парковке через личный кабинет на сайте Московского паркинга.

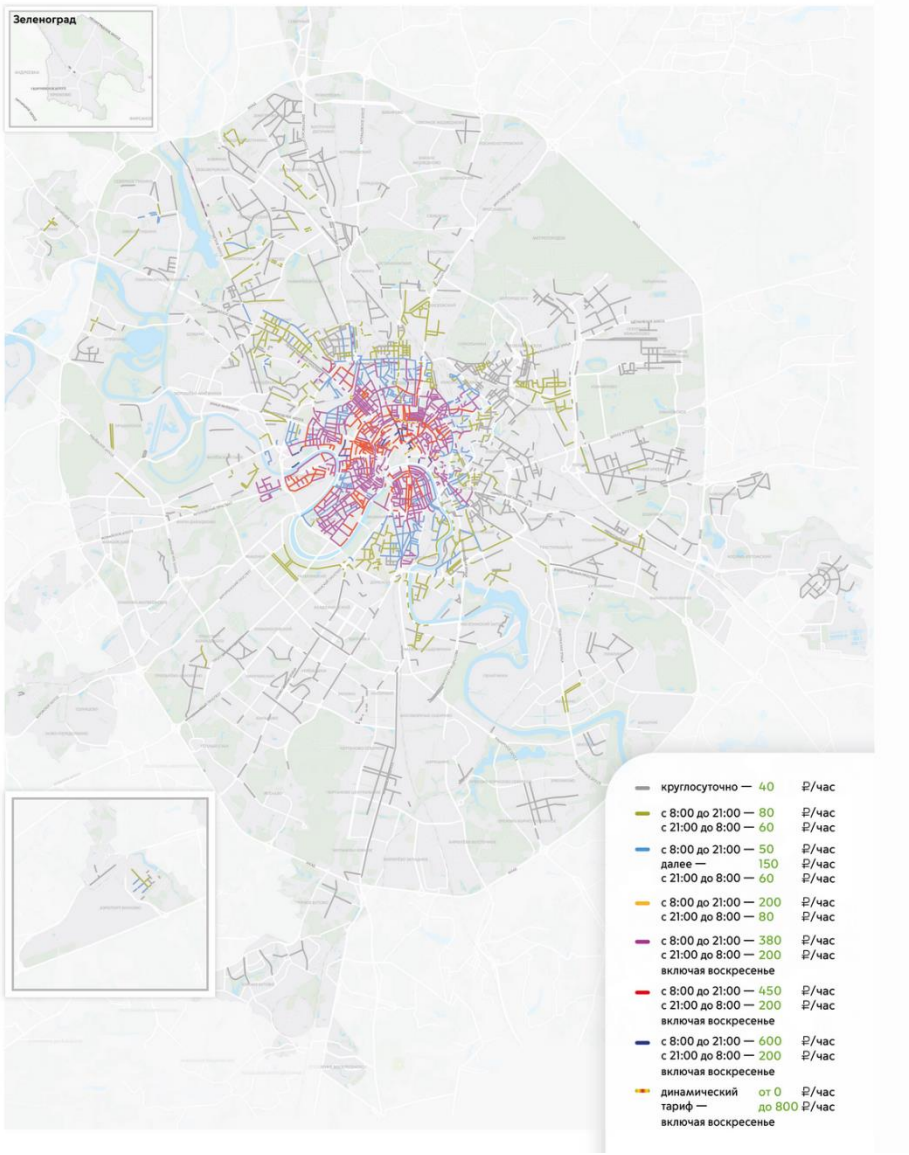


Рисунок 2 Карта парковочных мест

В качестве более удобного способа использования всех услуг данного сервиса лучше использовать приложение «Парковки». При регистрации пользователя в данном приложении необходимо ввести следующую информацию:

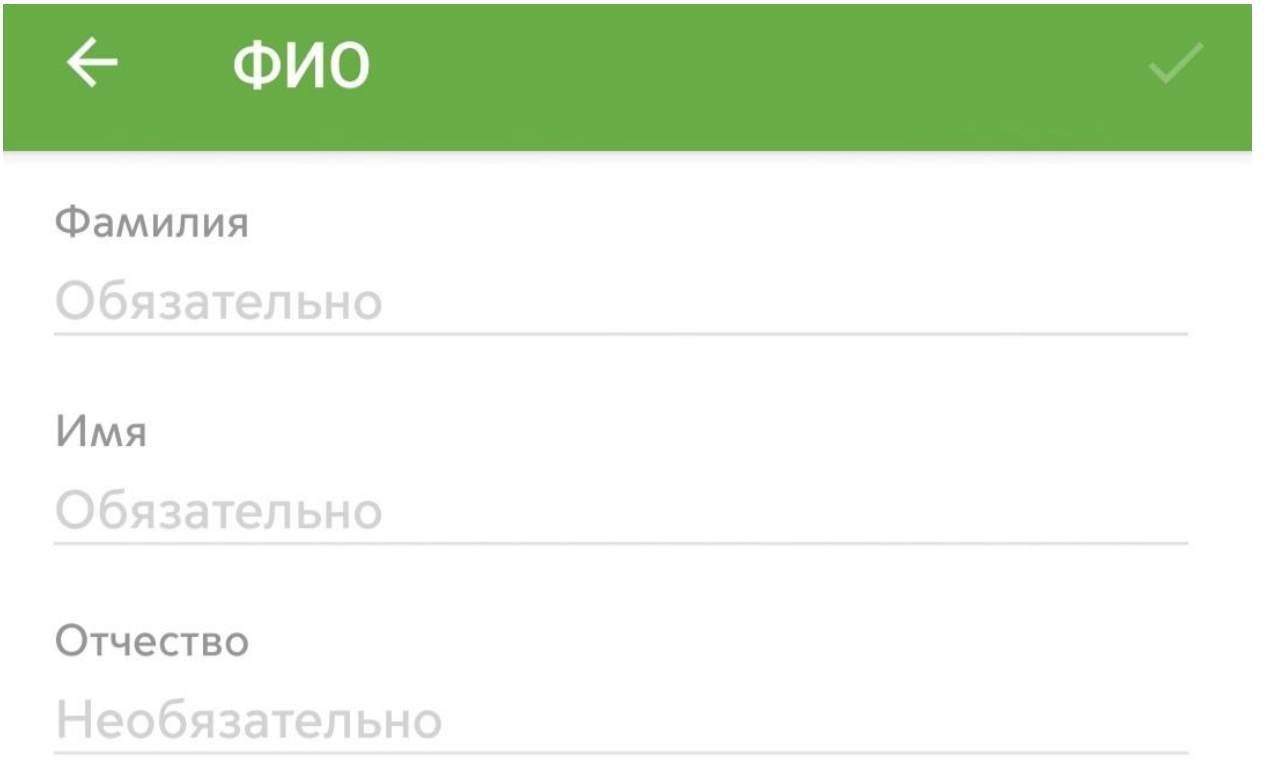


Рисунок 3.1 Внесение личных данных пользователя

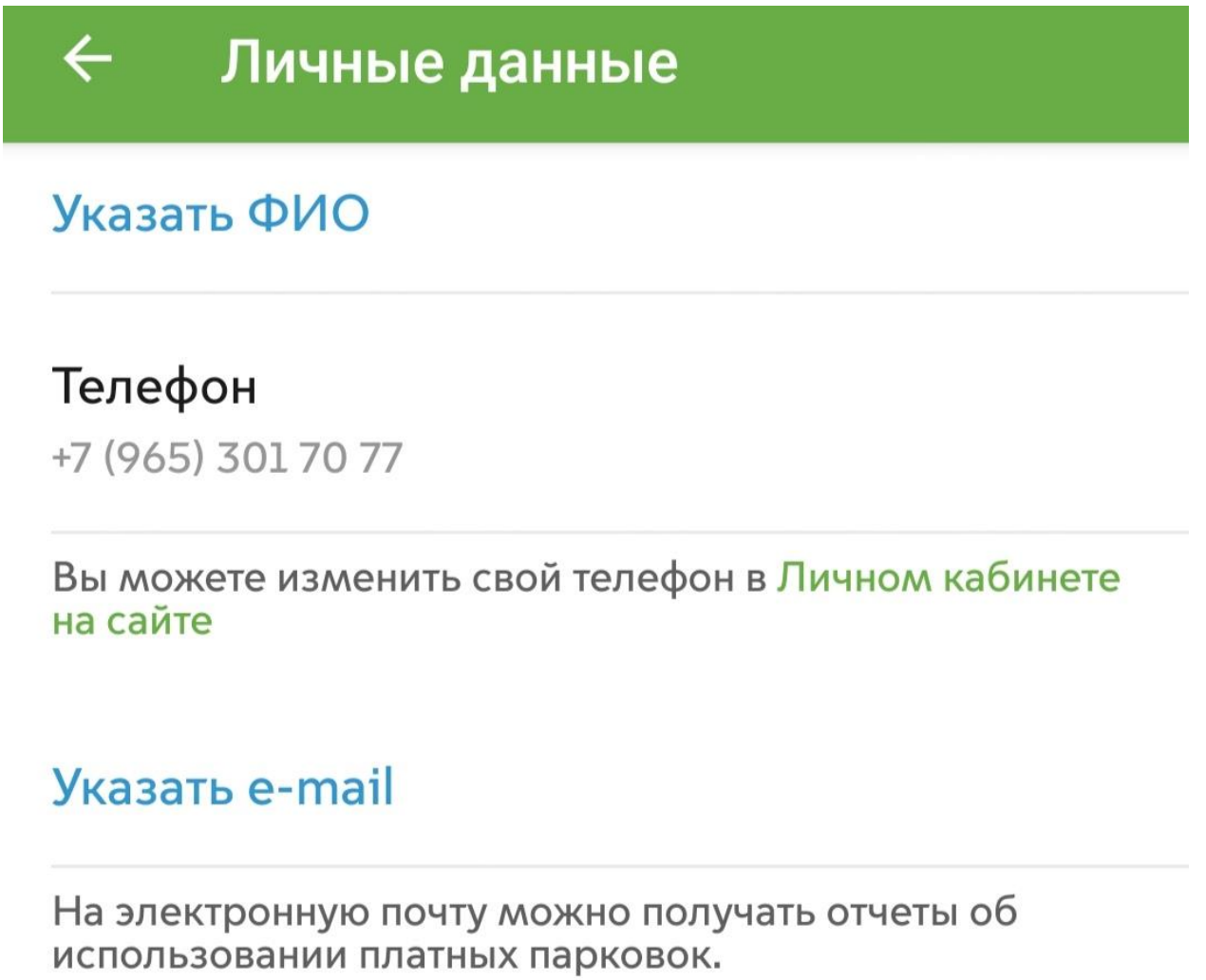


Рисунок 3.2 Внесение личных данных пользователя

После этого необходимо указать данные транспортного средств для его дальнейшей постановки на парковочное место и оплаты данной услуги

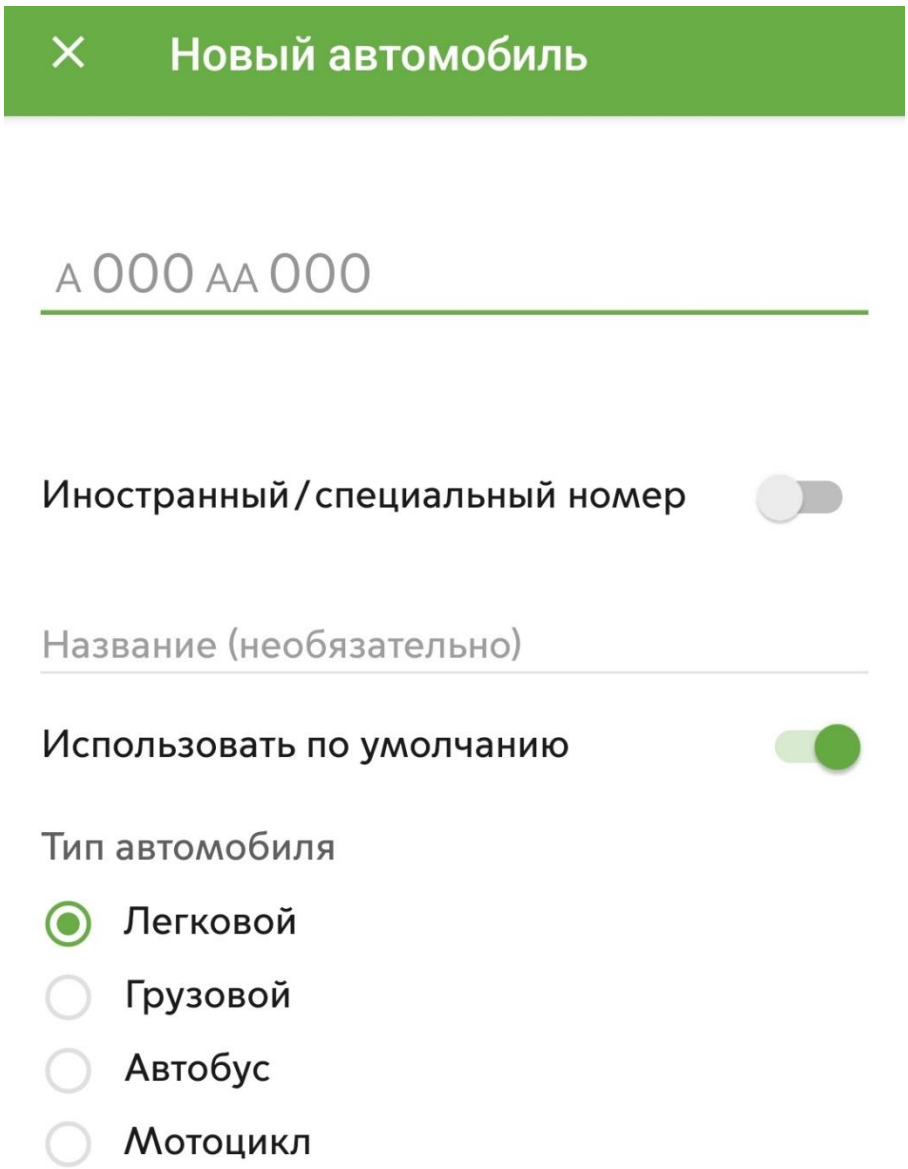


Рисунок 4 Внесение данных транспортного средства

На интерактивной карте пользователи могут видеть обозначения различных типов парковок, включая участки дорог, где разрешено или запрещено парковаться. Платные парковки выделены отдельно с указанием стоимости. [6] Благодаря этому пользователь может узнать информацию о парковочном месте

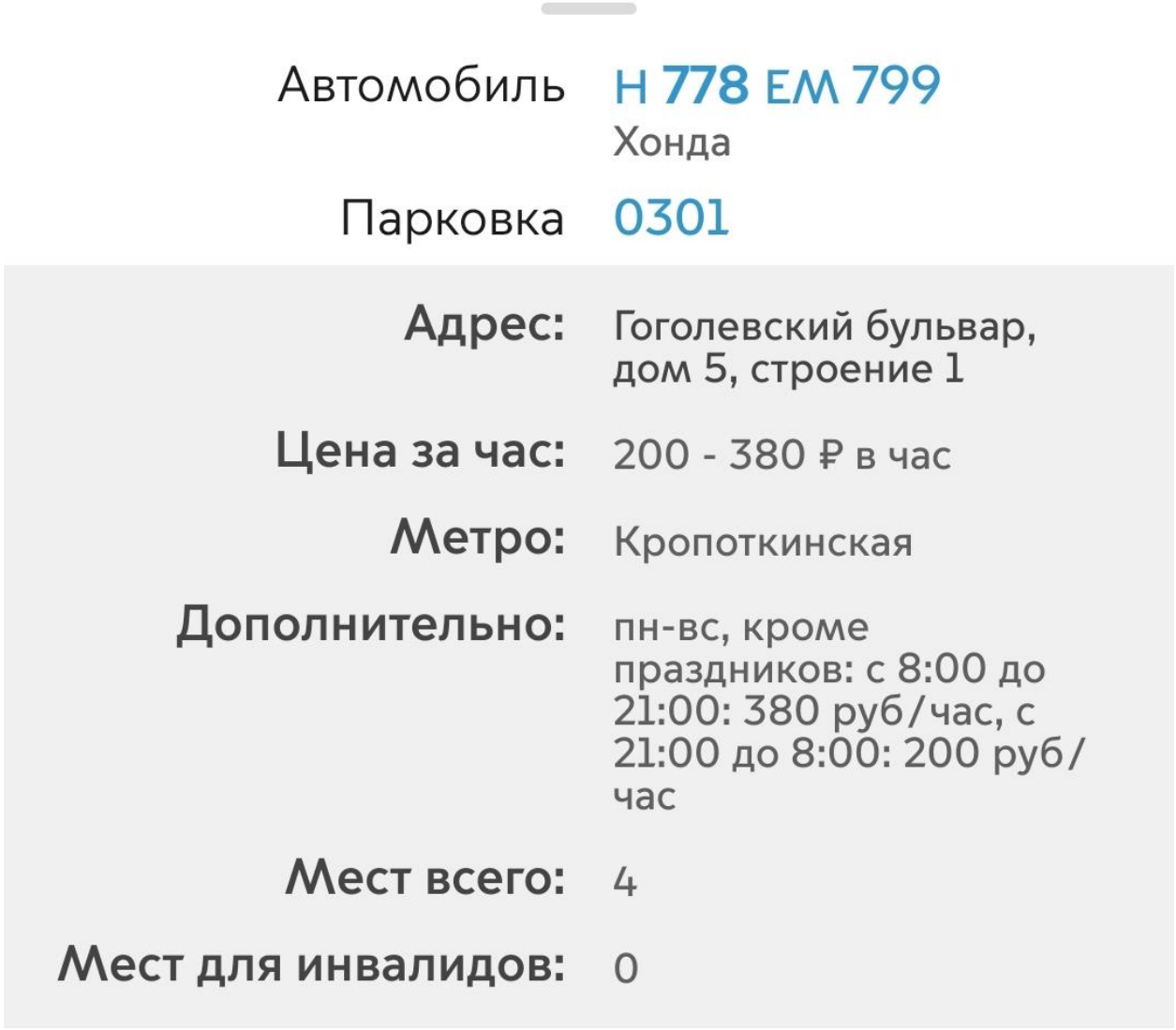


Рисунок 5 Информация о парковочном месте

Оплатить парковку можно несколькими способами [7]. В приложении процесс оплаты происходит следующим способом:

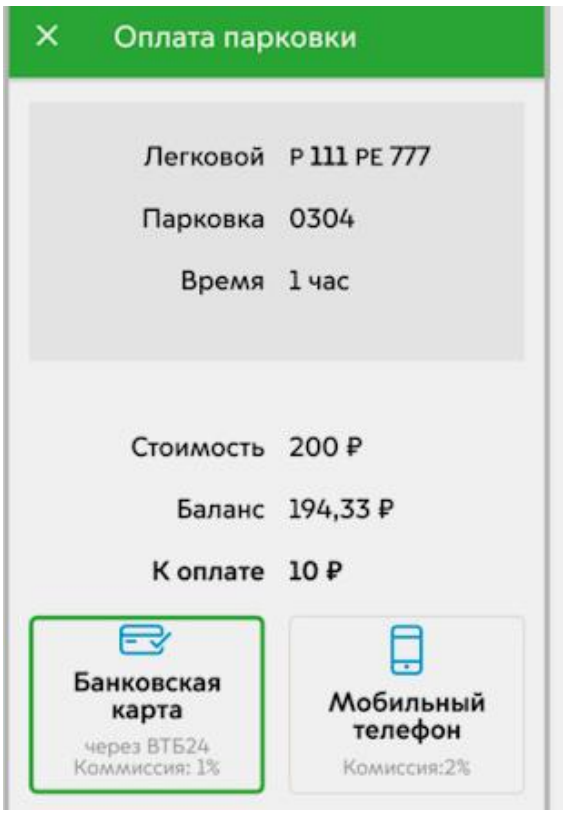


Рисунок 6 Оплата парковки

На сайте и в приложении полностью реализованы все функции, необходимые пользователю для эффективной работы с парковками. Это включает в себя возможность поиска доступных парковочных мест в режиме реального времени, бронирование парковочного места на определенный период, оплату парковки онлайн, получение уведомлений о времени окончания парковки, а также доступ к истории парковочных сессий. Все эти функции созданы с целью обеспечения максимального удобства и удовлетворенности пользователя при использовании сервиса.

1.2. Построение концептуальной модели в нотации Питера Чена

После анализа предметной области перейдем к построению концептуальной модели в нотации Питера Чена. Для этого выделим основные сущности в рассматриваемой предметной области, их атрибуты, а также их возможные связи друг с другом.

Работа по предоставлению услуги начинается с авторизации пользователя, значит выделяется сильная сущность «Пользователь», со следующим набором атрибутов: id (идентификатор), фамилия, имя, отчество, телефон, электронная почта.

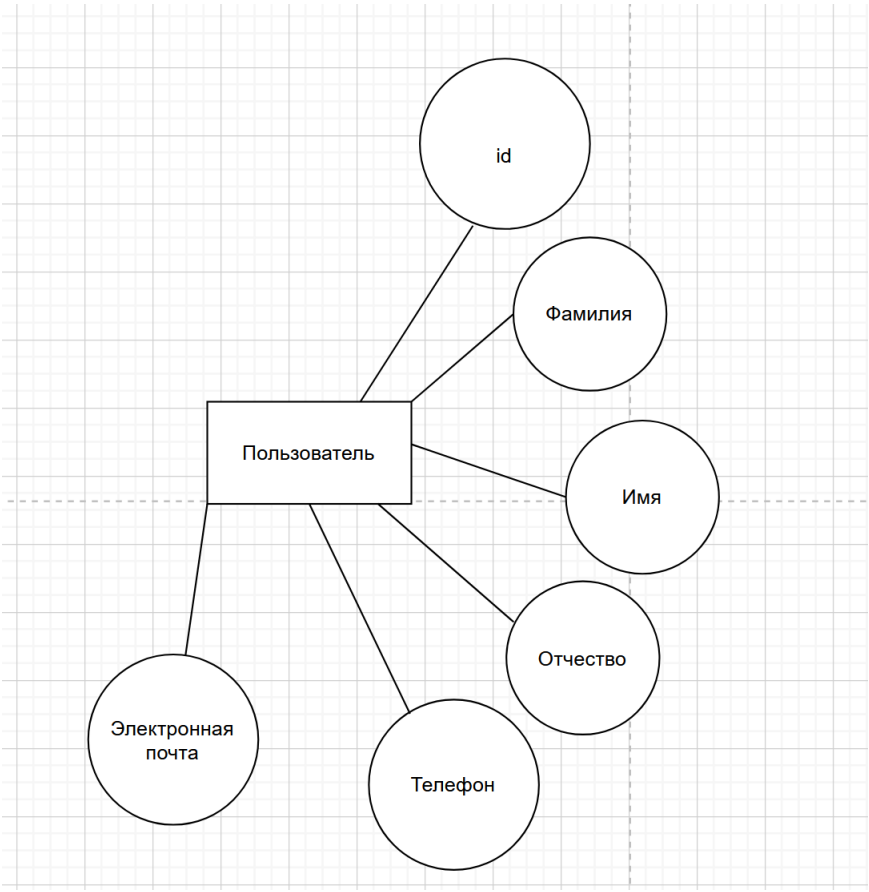


Рисунок 7 Сущность «Пользователь»

После регистрации пользователя, в базу данных вносится информация о транспортном средстве. Создадим новую сущность, «Транспорт», которая будет состоять из следующих атрибутов: название, id, название, номер транспорта, тип транспорта. Данная сущность являться слабой, так как информацию о транспорте вносит сам пользователь.

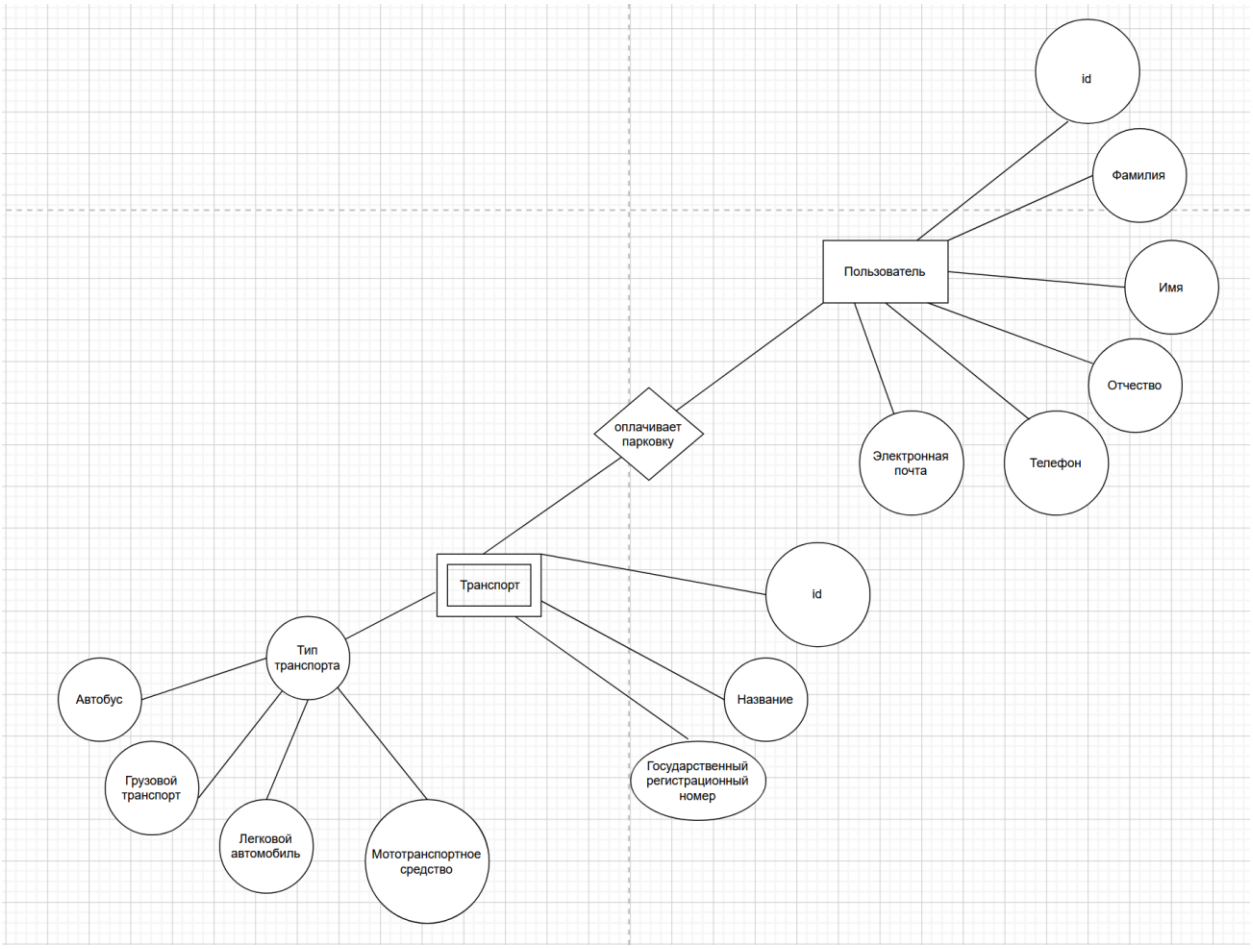


Рисунок 8 Сущности «Пользователь», «Транспорт»

Создадим следующую сущность: «Парковка», которая имеет следующие атрибуты: id парковки, дата парковки, время парковки. Данную сущность можно охарактеризовать как процесс учёта занятых парковочных мест транспортными средствами. В связи с этим можно сделать вывод, что сущность является зависимым от информации о транспортных средствах и парковочных местах.

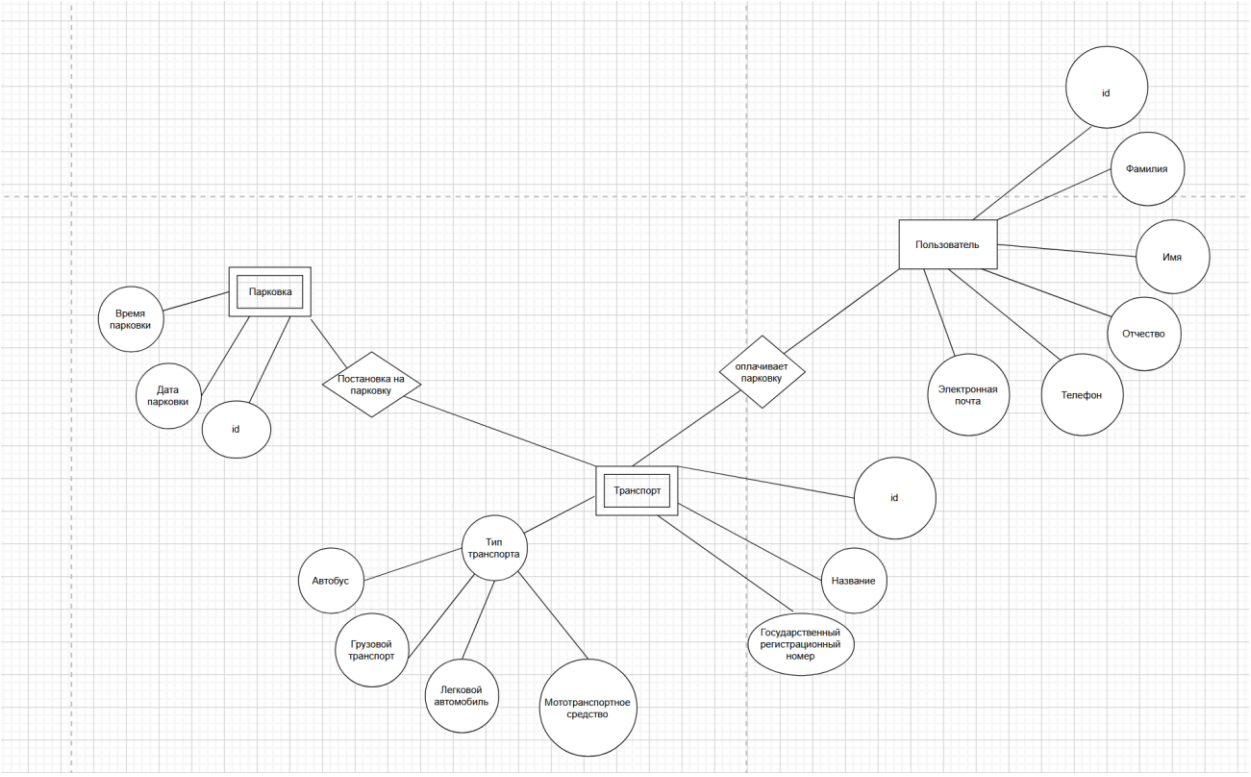


Рисунок 9 Сущности «Пользователь», «Транспорт», «Парковка»

Так как сущность «Парковка» зависит не только от транспорта, но и от базы парковочных мест, создадим следующую сущность: «Парковочное место», которая будет иметь следующие атрибуты: id, зона паркинга, стоимость парковки, тип места. Данная сущность является сильной, так как с неё будет считываться информация обо всех парковочных местах.

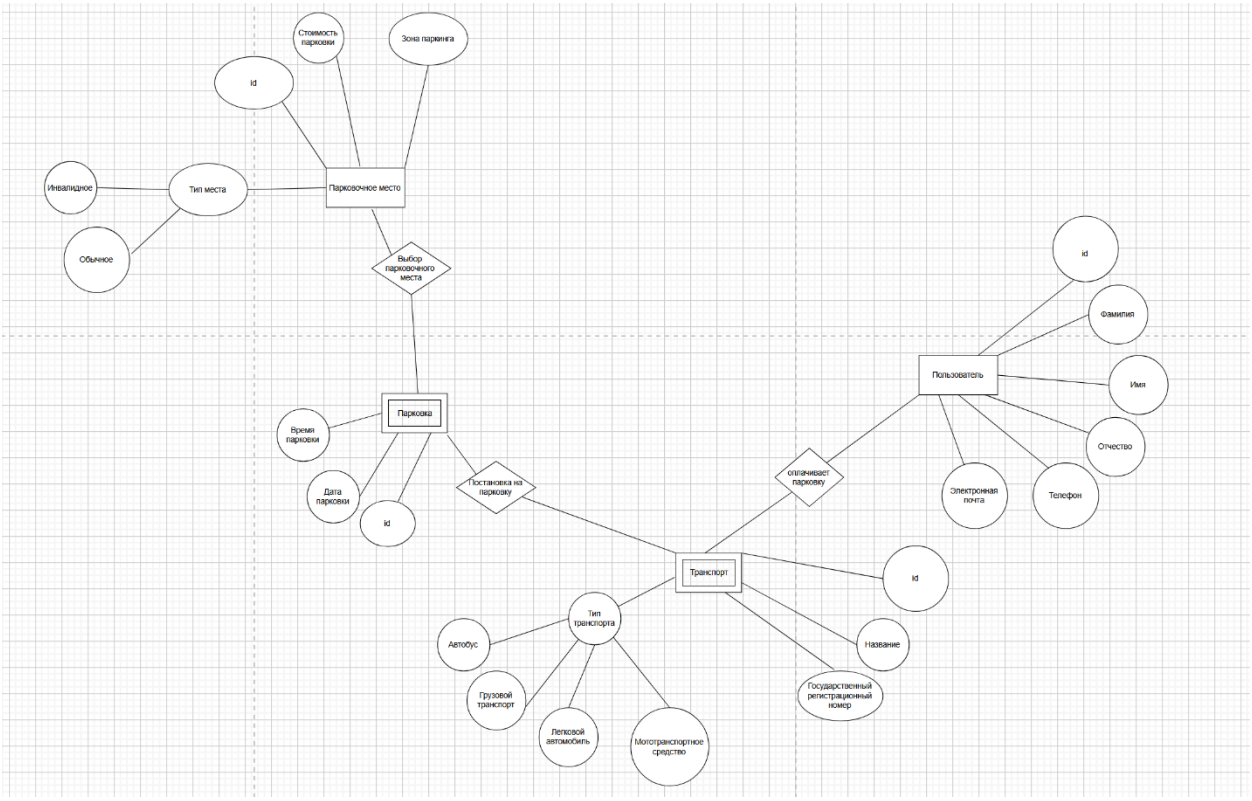


Рисунок 10 Сущности «Пользователь», «Транспорт», «Парковка», «Парковочное место»

После постановки на парковочное место транспортного средства необходимо произвести оплату за парковку. Для этого создадим новую сущность: «Платёж» с атрибутами: id, сумма платежа, дата платежа, способ оплаты, статус платежа. Данная сущность является слабой, так как зависит от того была произведена постановка транспорта на парковку или нет.

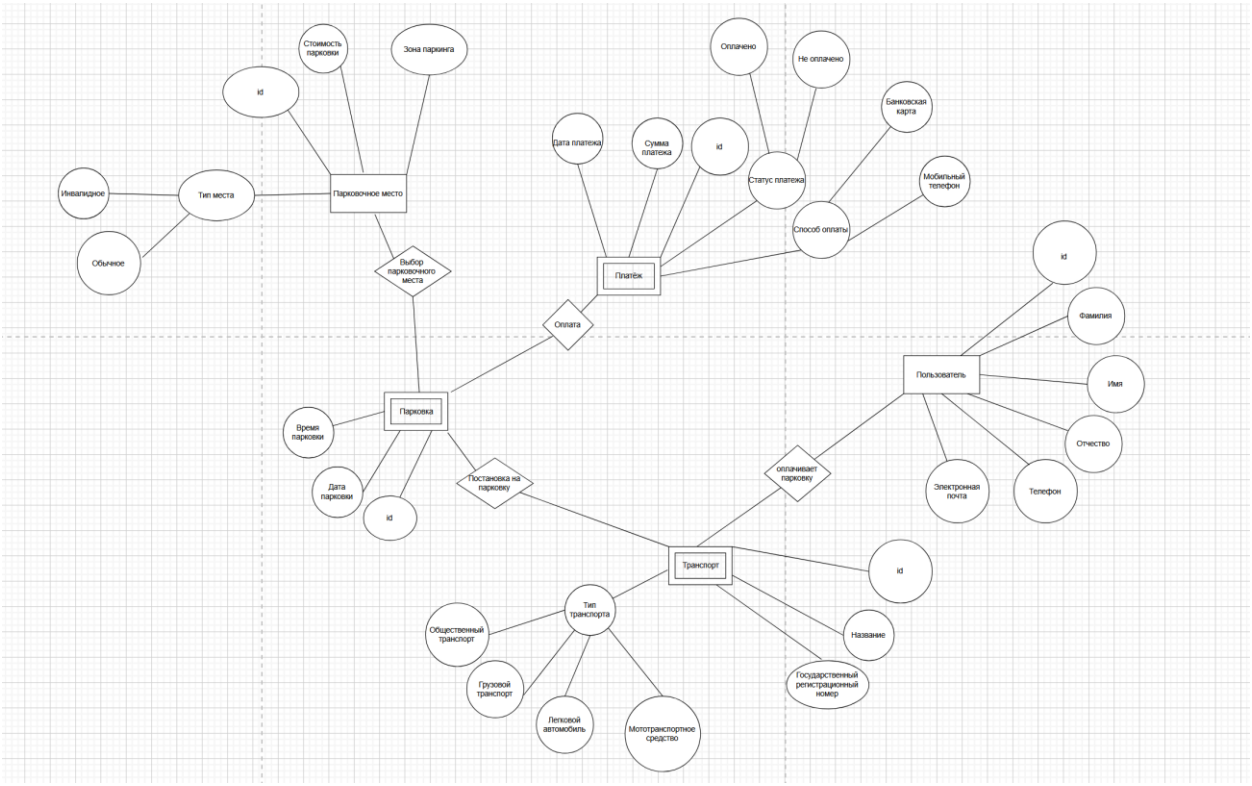


Рисунок 11 Сущности «Пользователь», «Транспорт», «Парковка», «Парковочное место», «Платёж»

Выводы по первой главе

Анализ существующей системы управления парковками в Москве выявил потребность в эффективной базе данных для управления парковочными местами, учета пользователей и их транспортных средств, а также оплаты услуг. На основе этого анализа была построена концептуальная модель базы данных в нотации Питера Чена, которая включает все необходимые сущности и связи между ними, закладывая фундамент для дальнейшего проектирования. Модель демонстрирует понимание предметной области и основных требований к системе.

РАЗДЕЛ 2. НОРМАЛИЗАЦИЯ И ПОСТРОЕНИЕ ДАТАЛОГИЧЕСКОЙ МОДЕЛИ

2.1. Нормализация. Построение даталогической модели

Перед построением даталогической модели базы данных, необходимо выполнить ряд действий: определение связей между сущностями, выбор вида базы данных, приведение модели базы данных к нормальным формам.

При дальнейшем проектирование и построении базы данных будет использована реляционная база данных – одна из наиболее распространенных видов баз данных.

Реляционные базы данных основаны на реляционной модели — интуитивно понятном, наглядном табличном способе представления данных. Каждая строка, содержащая в таблице такой базы данных, представляет собой запись с уникальным идентификатором, который называют ключом. Столбцы таблицы имеют атрибуты данных, а каждая запись обычно содержит значение для каждого атрибута, что дает возможность легко устанавливать взаимосвязь между элементами данных. [8]

Нормализация – это процесс последовательного преобразования таблиц базы данных к виду, принятому в реляционной модели данных. [13]

Специалисты различают пять последовательных этапов (форм) приведения данных к реляционному виду от первой нормальной формы (First Normal Form, 1NF) до пятой (5NF). На каждой из ступеней нормализации таблица приобретает новые черты, которые потребуются для перехода к очередному этапу. [14]

Для приведения базы данных к первой нормальной форме (1NF), необходимо соблюдение атомарности всех атрибутов таблиц. В данном случае, под атомарностью понимается целостность и неделимость данных в базе данных. Для удобства в каждой реляционной таблице будем использовать только суррогатные простые первичные ключи, это позволит автоматически

обеспечить 2 NF и соответственно полностью пропустить этот этап нормализации.

Сущность «Пользователь» – таблица «User»

Определим первичный ключ **id**, типа значения INT.

Атрибут «Фамилия» обозначим **Surname**, с типом данных STRING.

Атрибут «Имя» обозначим **Name**, с типом данных STRING.

Атрибут «Отчество» обозначим **Patronym**, с типом данных STRING.

Атрибут «Телефон» обозначим **phone**, с типом данных VARCHAR(11).

Атрибут «Электронная почта» обозначим **email**, с типом данных STRING.

Все атрибуты, включая первичный ключ не могут быть пустыми, поэтому имеют ограничение NOT NULL.

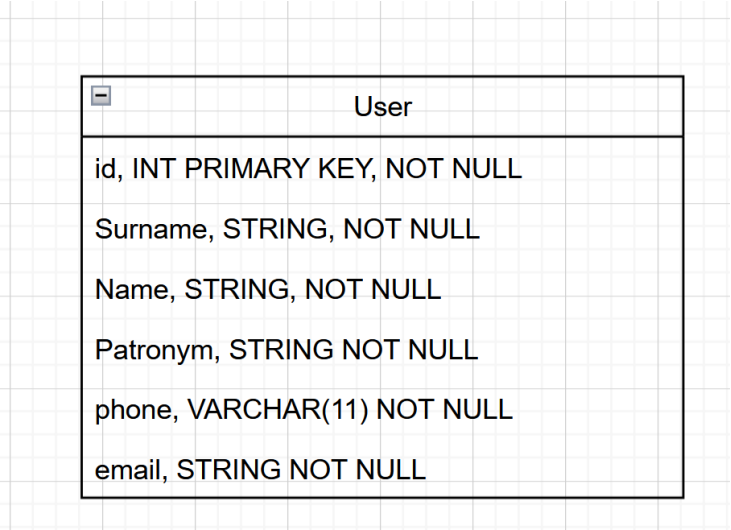


Рисунок 12 Реляционная таблица сущности «Пользователь»

Сущность «Транспортное средство» – таблица «Transport»

Определим первичный ключ **id**, типа значения INT.

Атрибут «Идентификатор пользователя» обозначим **User_id**. Чтобы использовать данное поле как внешний ключ справочной таблицы, обозначим его тип как INT.

Атрибут «Название» обозначим **name**, с типом данных STRING.

Атрибут «Номер транспортного средства» обозначим **number_transport**, с типом данных VARCHAR(8).

Атрибут «Тип транспортного средства» обозначим **type_transport_id**, чтобы использовать данное поле как внешний ключ справочной таблицы, обозначим его тип как INT.

Все атрибуты, включая первичный ключ не могут быть пустыми, поэтому имеют ограничение NOT NULL.

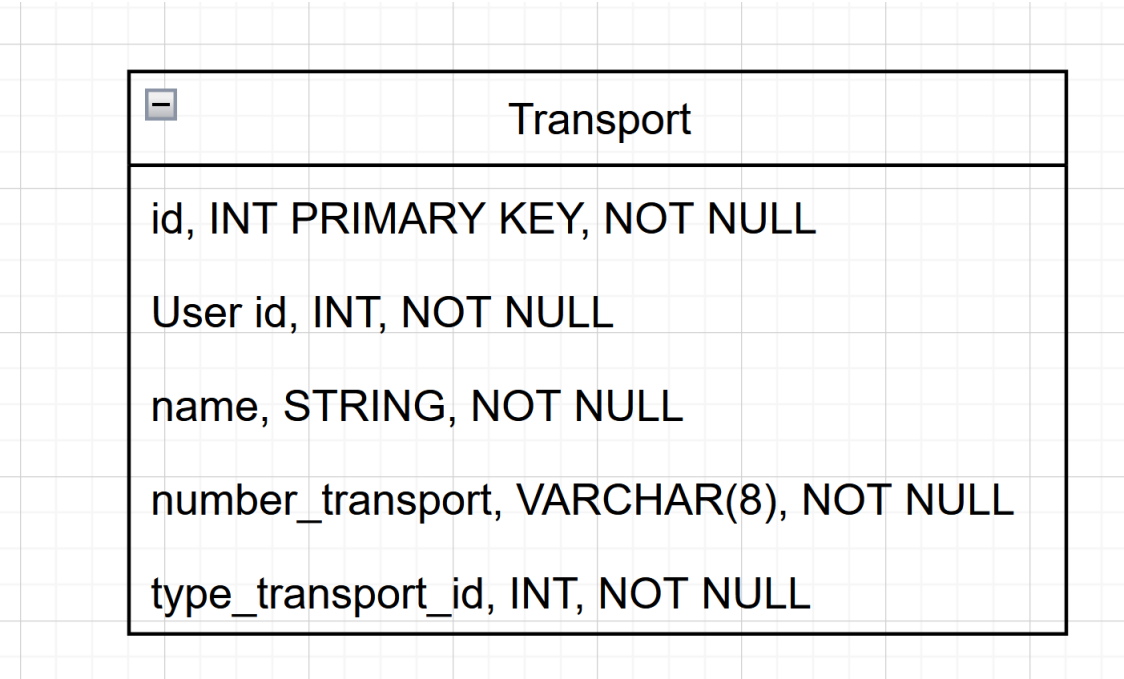


Рисунок 13 Реляционная таблица сущности «Транспорт»

Сущность «Парковка» – таблица «Parking»

Определим первичный ключ **id**, типа значения INT.

Атрибут «Идентификатор парковочного места» обозначим **Parking_space_id**. Чтобы использовать данное поле как внешний ключ справочной таблицы, обозначим его тип как INT.

Атрибут «Идентификатор транспортного средства» обозначим **Transport_id**. Чтобы использовать данное поле как внешний ключ справочной таблицы, обозначим его тип как INT.

Атрибут «Время парковки» обозначим **Parking_time**, с типом данных STRING (указывается строка формата ЧЧ:ММ).

Атрибут «Дата парковки» обозначим **Parking_date**, с типом данных DATE.

Все атрибуты, включая первичный ключ не могут быть пустыми, поэтому имеют ограничение NOT NULL.

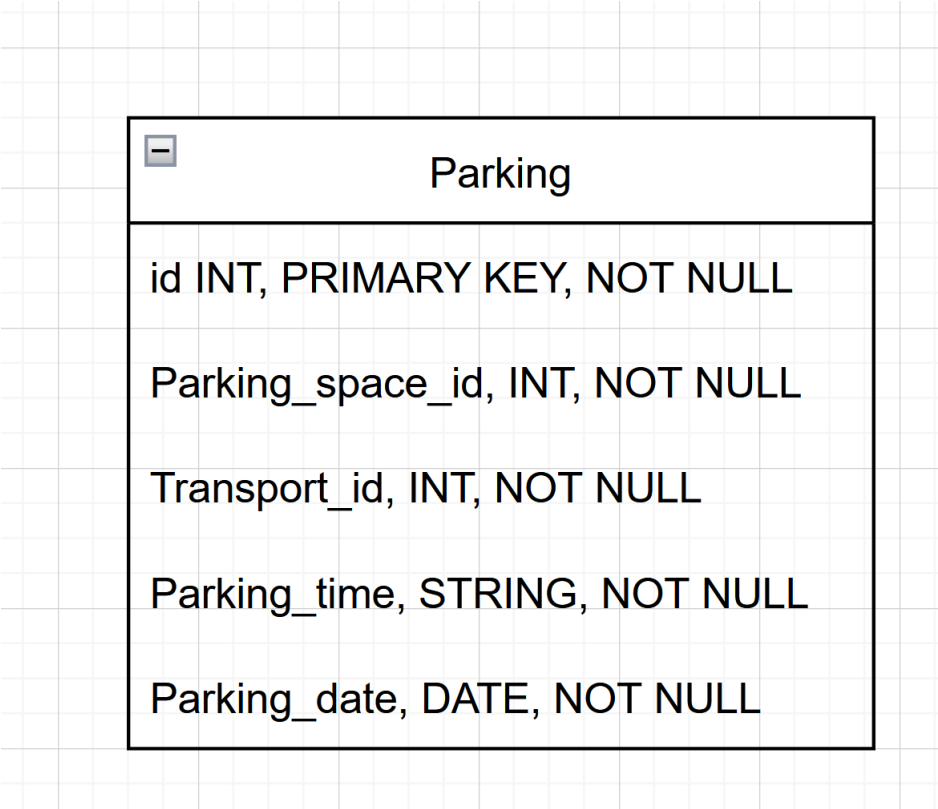


Рисунок 14 Реляционная таблица сущности «Парковка»

Сущность «Парковочное место» – таблица «Parking space»

Определим первичный ключ **id**, типа значения INT.

Атрибут «Зона парковки» обозначим **Parking_area**, с типом данных VARCHAR(80).

Атрибут «Тип парковочного места» обозначим **type_place_id**. Чтобы использовать данное поле как внешний ключ справочной таблицы, обозначим его тип как INT.

Атрибут «Стоимость парковки» обозначим **Parking_cost**, с типом данных FLOAT (указывается сумма с копейками).

Все атрибуты, включая первичный ключ не могут быть пустыми, поэтому имеют ограничение NOT NULL.

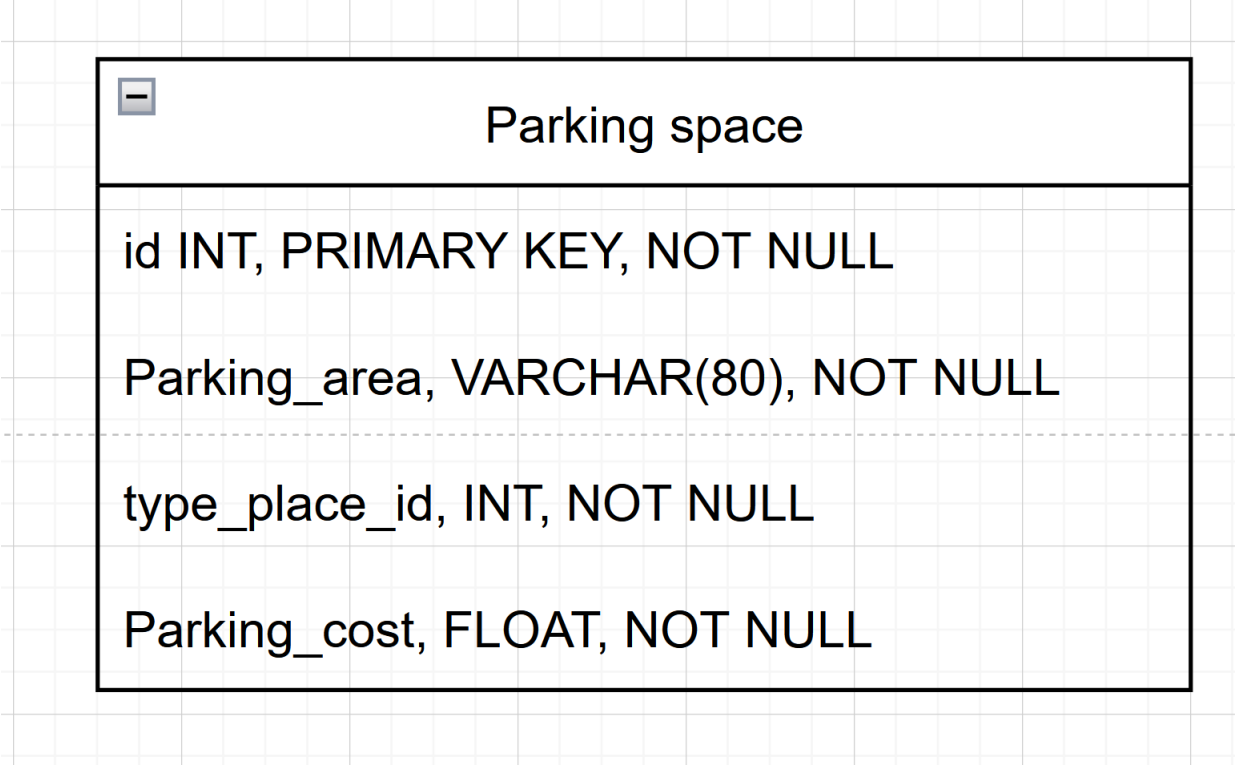


Рисунок 15 Реляционная таблица сущности «Парковочное место»

Сущность «Платеж» – таблица «Payment»

Определим первичный ключ **id**, типа значения INT.

Атрибут «Идентификатор парковки» обозначим **Parking_id**. Чтобы использовать данное поле как внешний ключ справочной таблицы, обозначим его тип как INT.

Атрибут «Сумма платежа» обозначим **Payment_amount**, с типом данных FLOAT.

Атрибут «Дата платежа» обозначим **Payment_date**, с типом данных DATE.

Атрибут «Способ платежа» обозначим **payment_method_id**. Чтобы использовать данное поле как внешний ключ справочной таблицы, обозначим его тип как INT.

Атрибут «Статус платежа» обозначим **payment_status_id**. Чтобы использовать данное поле как внешний ключ справочной таблицы, обозначим его тип как INT.

Все атрибуты, включая первичный ключ не могут быть пустыми, поэтому имеют ограничение NOT NULL.

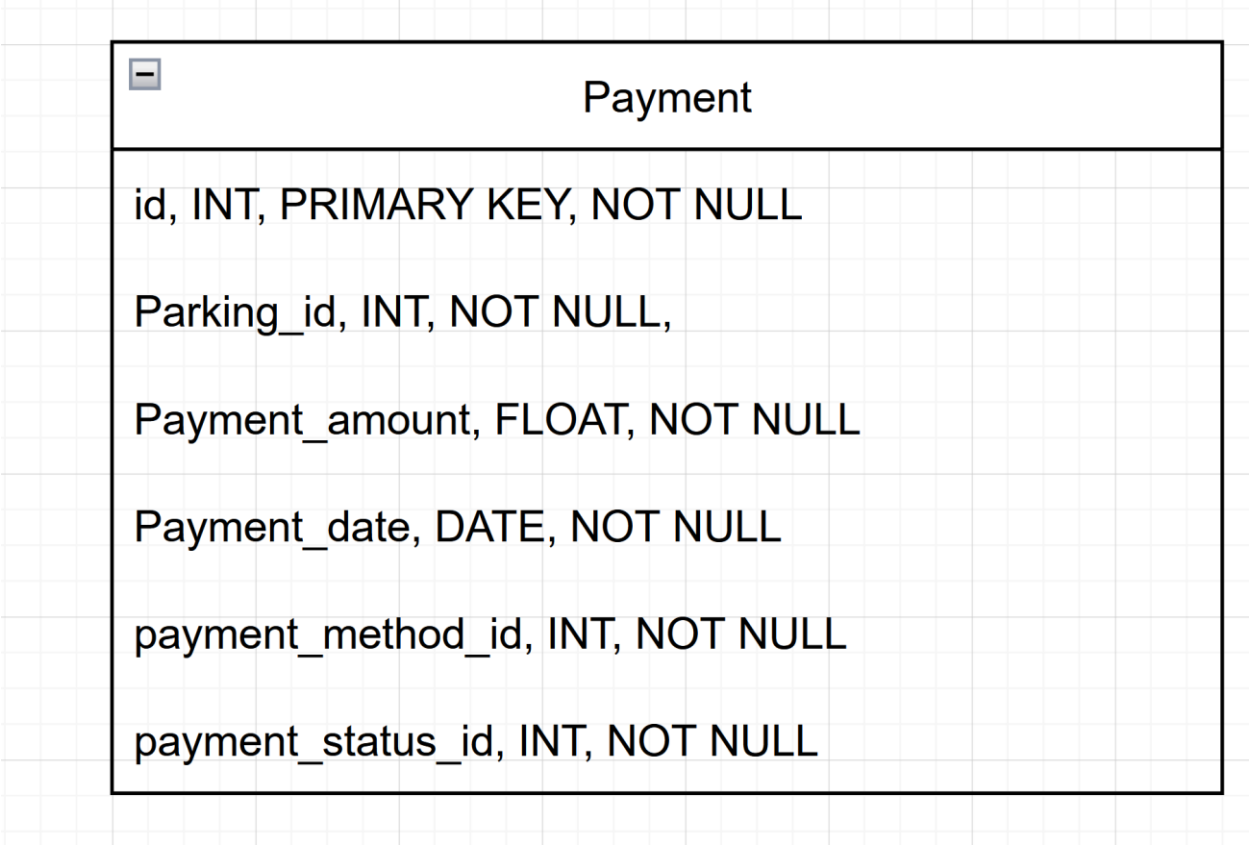


Рисунок 16 Реляционная таблица сущности «Платёж»

Таким образом, этап приведения к 1NF завершен.

Приведение ко второй нормальной форме направлено на устранение частичных зависимостей, которые могут возникать между частями составного первичного ключа и остальными атрибутами. Поскольку на предыдущем этапе

первичные ключи всех таблиц были выбраны простыми, база данных уже приведена ко второй нормальной форме.

Таблица находится в третьей нормальной форме 3NF, если между ее атрибутами нет транзитивных связей (внутри приведенной к 3NF таблицы должны отсутствовать связи «один ко многим»).

Таблица приведена к четвертой нормальной форме, если она соответствует 3NF и в ней отсутствуют многозначные зависимости (многие ко многим).

Для приведения таблиц к 3NF и 4NF, необходимо реализовать связи между сущностями.

Определение связи между таблицами «User», и «Transport». При регистрации пользователь вносит свои данные, а также данные своего транспортного средства. В приложении присутствует функция повторного добавления информации о новом транспортном средстве. Благодаря данной функции связь между таблицами можно охарактеризовать «один к многим», т.е необходимо добавить в таблицу «Transport» внешний ключ, с обозначением **User_id**, тип данных INT.

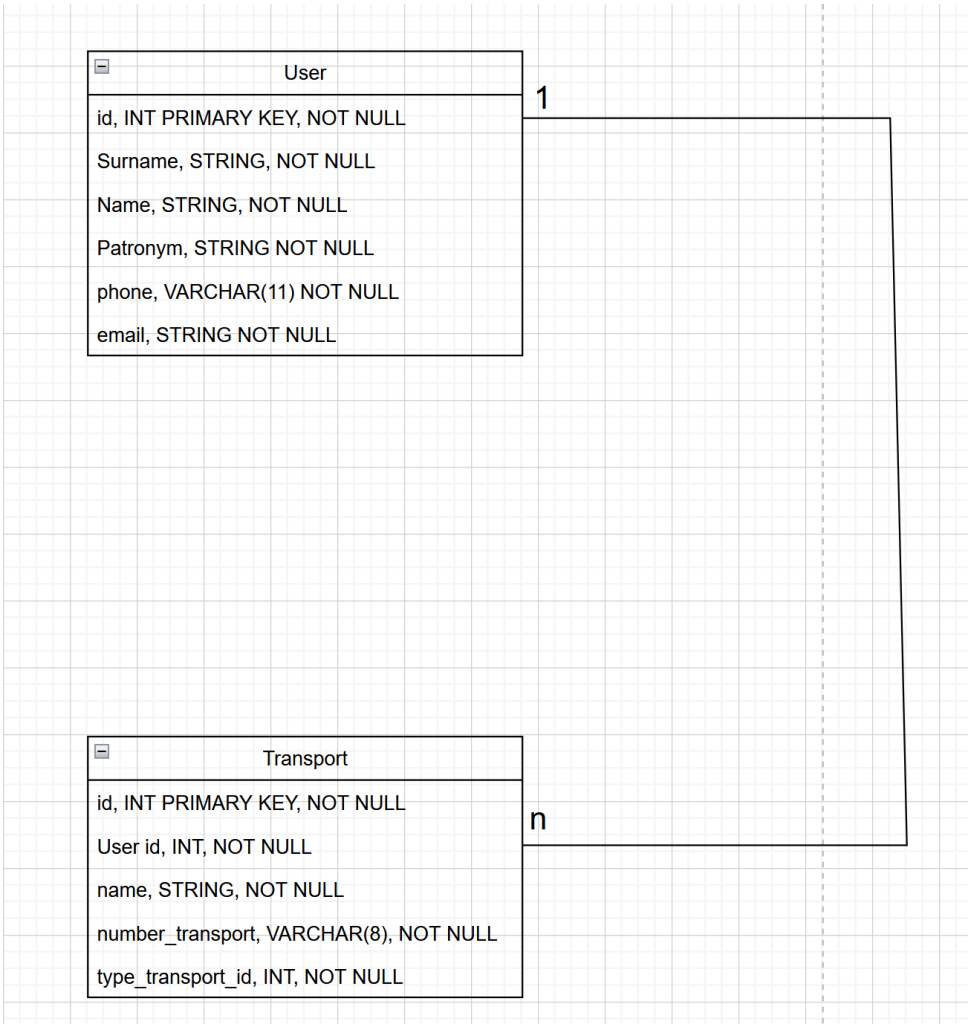


Рисунок 17 Связь между сущностями «Пользователь» – «Транспорт»

При выборе парковочного места и дальнейшей постановке на парковку транспортного средства формируется связь между таблицами: Transport, Parking, Parking space. Так как парковочных мест может быть большое количество, транспортное средство можно ставить на парковочное место неоднократно, между таблицами: Parking space и Parking формируется связь «один к многим» с последующим добавлением в таблицу Parking внешнего ключа: **Parking_space_id**, тип данных INT. Между таблицами Transport и Parking тоже формируется связь «Один ко многим» с последующим добавлением в таблицу Parking внешнего ключа **Transport_id**, тип данных INT.

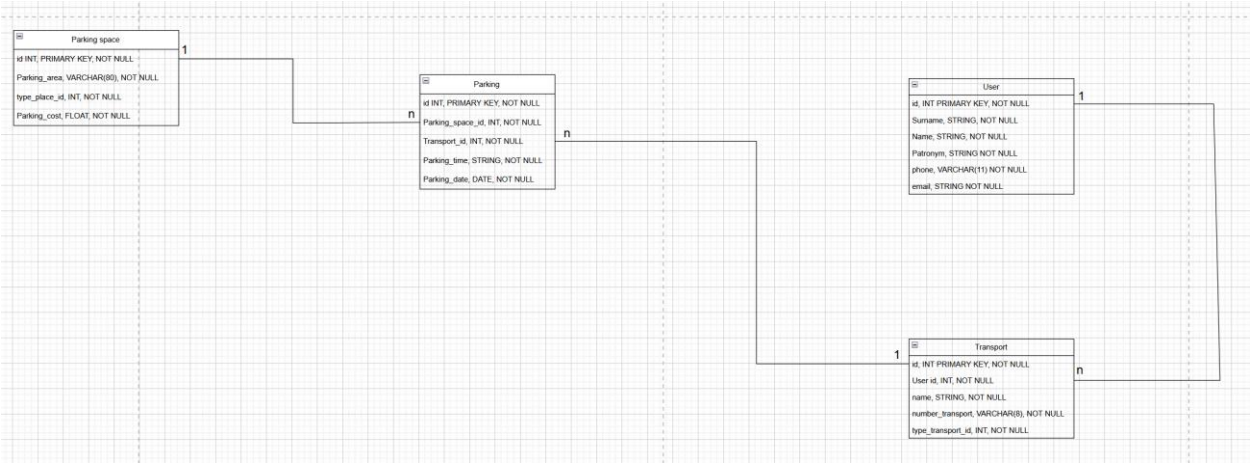


Рисунок 18 Связь между сущностями: «Транспорт», «Парковка», «Парковочное место»

После того как транспортное средство будет припарковано, необходимо оплатить стоимость парковки. Так как платежная операция за парковочное место может производиться неоднократно, между таблицами Parking и Payment формируется связь «Один ко многим» с последующим добавлением в таблицу Payment внешнего ключа **Parking_id**, тип данных INT.

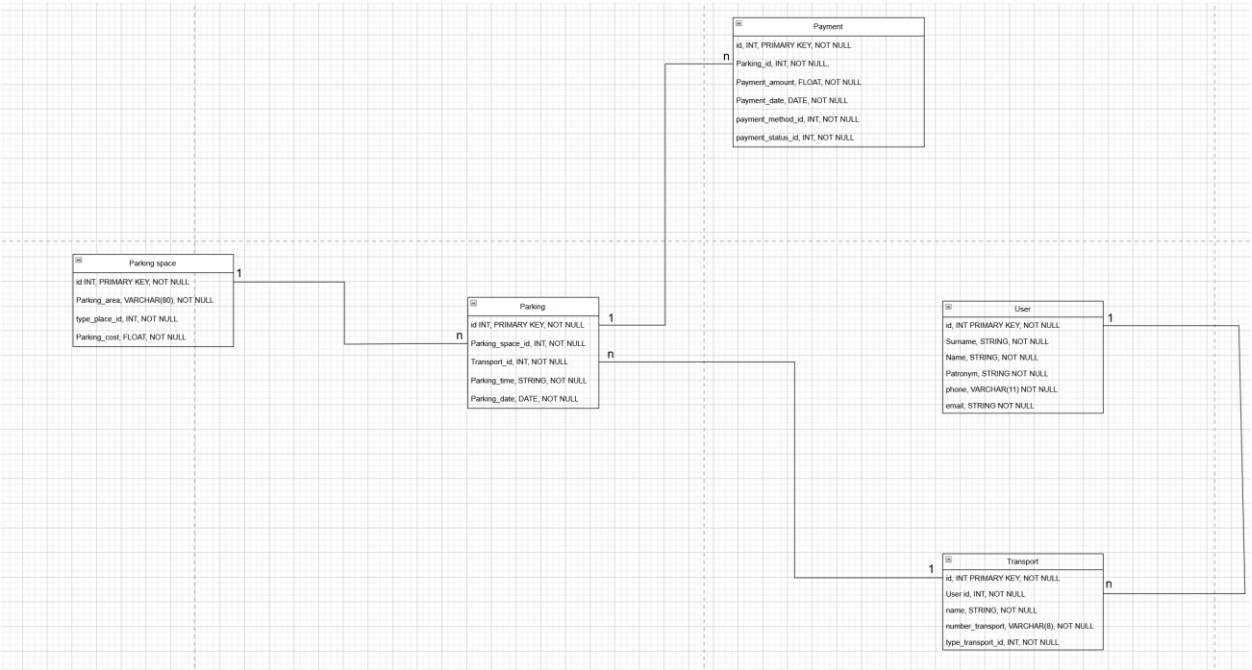


Рисунок 19 Основные связи между сущностями

2.2 Добавление справочных таблиц

Для завершения этапа даталогического проектирования остается рассмотреть структуру и наполнение справочных таблиц: **type transport, type place**, **payment method**, **payment status**.

Справочная таблица type transport включает в себя следующие поля:

Первичный ключ таблицы **id**, тип значения INT, поле **type**, тип значения STRING. Таблица содержит следующие записи: *легковой автомобиль*, *мототранспортное средство*, *грузовой транспорт, общественный транспорт*.

Справочная таблица type_place включает в себя следующие поля:

Первичный ключ таблицы **id**, тип значения INT, поле **type**, тип значения STRING. Таблица содержит следующие записи: *обычное, инвалидное.*

Справочная таблица payment_method включает в себя следующие поля:

Первичный ключ таблицы **id**, тип значения INT, поле **method**, тип значения STRING. Таблица содержит следующие записи: *банковская карта*, *наличные.*

Справочная таблица **payment_status** включает в себя следующие поля:

Первичный ключ таблицы **id**, тип значения INT, поле **status**, тип значения STRING. Таблица содержит следующие записи: *оплачено, не оплачено.*

Окончательный вид даталогической модели представлен на рисунке 20.

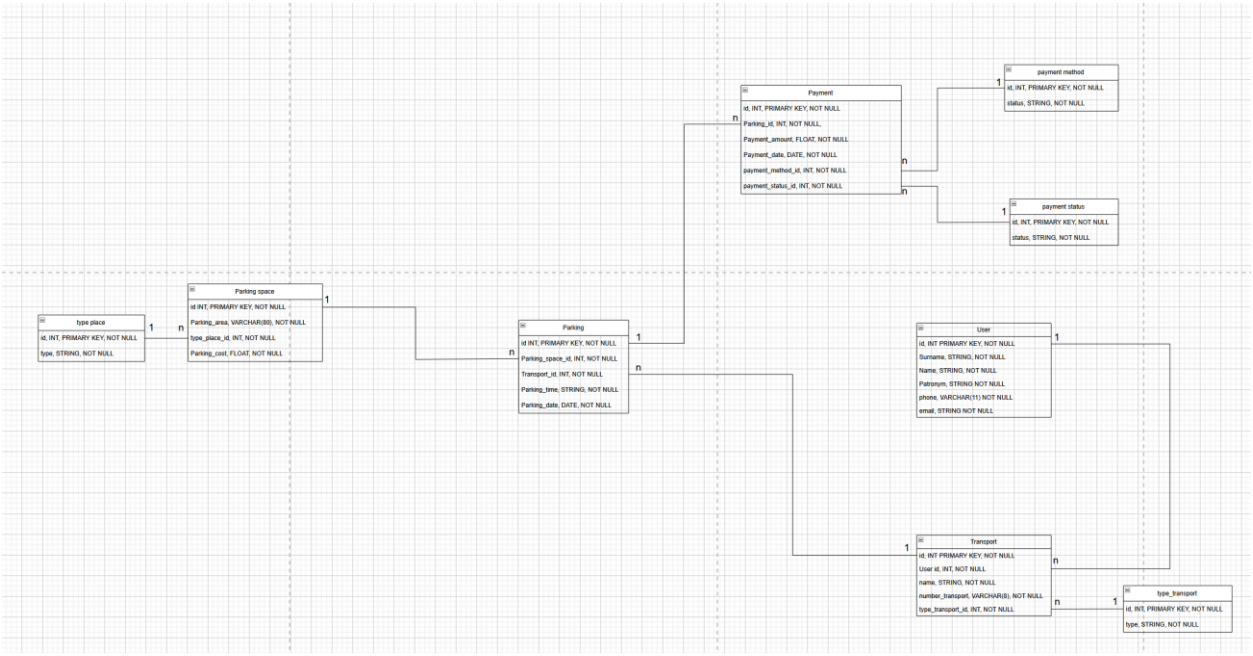


Рисунок 20 Датологическая модель базы данных предметной области

Выводы по второй главе

Концептуальная модель была успешно нормализована до третьей нормальной формы (3NF), что устранило избыточность данных и обеспечило целостность информации. Были определены первичные и внешние ключи, типы данных для всех атрибутов, а также созданы справочные таблицы для повышения эффективности и удобства работы с базой данных. Результат этой главы - структурированная и оптимизированная даталогическая модель, готовая к реализации в физической базе данных.

РАЗДЕЛ 3. ПОСТРОЕНИЕ ФИЗИЧЕСКОЙ МОДЕЛИ БАЗЫ ДАННЫХ

3.1. Создание таблиц базы данных

Построение физической модели базы данных также включает в себя выбор конкретной системы управления базой данных (СУБД). Для построения в данной работе будет использована библиотека SQLite, которая является одной из самых широко-используемых СУБД в мире, которую используют такие крупные компании, как Adobe, Apple, Google и другие. [9-10]

Первый этап построения физической модели включает в себя создание пустой базы данных на сервере СУБД. Помимо использования библиотеки sqlite3, для работы в среде colab, будет использован язык программирования python.

В результате выполнения программного кода, представленного в листинге 1, создается файл базы данных.

Листинг 1. Создание базы данных **Berestenkov.db**

```
import sqlite3 # импортируем библиотеку по работе с базой данных
try:
sqlite_connection = sqlite3.connect('Berestenkov.db') #создаем переменную подключения к БД
    cursor = sqlite_connection.cursor() #создаем специальную переменную по работе с БД
    print("База данных создана и успешно подключена к SQLite")

    sqlite_select_query = "SELECT sqlite_version();" #пишем запрос получение версии
  библиотеки sqlite
    cursor.execute(sqlite_select_query) #выполняем запрос, после чего в буфере содержится
  текст результата
    record = cursor.fetchall() #читаем все содержимое буфера в переменную record
    print("Версия базы данных SQLite: ", record)
    cursor.close() #закрываем курсор по работе с БД

except sqlite3.Error as error:
    print("Ошибка при подключении к sqlite", error) # если при попытке подключения к БД
  возникли ошибки, выводим их
finally:
    if (sqlite_connection):
        sqlite_connection.close() #если соединение с БД открыто, закрываем его
        print("Соединение с SQLite закрыто")
База данных создана и успешно подключена к SQLite
Версия базы данных SQLite:  [('3.37.2',)]
Соединение с SQLite закрыто
```

После создания базы данных можно приступать к созданию и заполнению таблиц. Вначале целесообразно создать все справочные таблицы,

поскольку они, как правило, не зависят от наличия других таблиц. В нашем случае таких таблиц 4.

В результате выполнения программного кода, представленного в листинге 2, создаются таблицы: **type_transport, type_place**, **payment_method**, **payment_status**, все они имеют одинаковую структуру.

Листинг 2. Создание справочных таблиц

```
import sqlite3
try:
    sqlite_connection = sqlite3.connect('Berestenkov.db')
# Создаем sql-запросы на создание таблиц, записываем для удобства их в переменные с одноименным названием

# Создаём таблицу Тип автомобиля
    type_transport = 'CREATE TABLE type_transport (id INTEGER PRIMARY KEY NOT NULL, type string NOT NULL);'

# Создаём таблицу Тип места
    type_place = 'CREATE TABLE type_place (id INTEGER PRIMARY KEY NOT NULL, type string NOT NULL);'

# Создаём таблицу Способы оплаты
    payment_method = 'CREATE TABLE payment_method (id INTEGER PRIMARY KEY NOT NULL, method string NOT NULL);'

# Создаём таблицу Статус платежа
    payment_status = 'CREATE TABLE payment_status (id INTEGER PRIMARY KEY NOT NULL, status string NOT NULL);'

    cursor = sqlite_connection.cursor()
    print("База данных подключена к SQLite")
    cursor.execute(type_transport)
    sqlite_connection.commit()
    print("Таблица type_transport создана")

    cursor.execute(type_place)
    sqlite_connection.commit()
    print("Таблица type_place создана")

    cursor.execute(payment_method)
    sqlite_connection.commit()
    print("Таблица payment_method создана")

    cursor.execute(payment_status)
    sqlite_connection.commit()
    print("Таблица payment_status создана")

    cursor.close()

except sqlite3.Error as error:
    print("Ошибка при подключении к sqlite", error)
finally:
    if (sqlite_connection):
        sqlite_connection.close()
        print("Соединение с SQLite закрыто")

```

```
Соединение с SQLite закрыто
```

После создания справочных таблиц необходимо реализовать запросы на заполнение справочных таблиц (листинги 3-6).

Листинг 3. Заполнение таблицы **type_transport**

```
# Заполнение таблицы type_transport
import sqlite3
try:
    sqlite_connection = sqlite3.connect('Berestenkov.db')
    type_transport = '''INSERT INTO type_transport (id, type) VALUES
                        (1, 'Легковой автомобиль'),
                        (2, 'Мототранспортное средство'),
                        (3, 'Грузовой транспорт'),
                        (4, 'Общественный транспорт');
                    '''
    cursor = sqlite_connection.cursor()
    cursor.execute(type_transport)
    sqlite_connection.commit()
    cursor.close()
except sqlite3.Error as error:
    print("Ошибка при подключении к sqlite", error)
finally:
    if (sqlite_connection):
        sqlite_connection.close()
        print("Соединение с SQLite закрыто")
Соединение с SQLite закрыто
```

Листинг 4. Заполнение таблицы **type_place**

```
Заполнение таблицы type_place
import sqlite3
try:
sqlite_connection = sqlite3.connect('Berestenkov.db')
type_place = '''INSERT INTO type_place (id, type) VALUES
             (1, 'Обычное'),
             (2, 'Инвалидное');
              '''
cursor = sqlite_connection.cursor()
cursor.execute(type_place)
sqlite_connection.commit()
cursor.close()
except sqlite3.Error as error:
print("Ошибка при подключении к sqlite", error)
finally:
if (sqlite_connection):
sqlite_connection.close()
print("Соединение с SQLite закрыто")
Соединение с SQLite закрыто
```

Листинг 5. Заполнение таблицы **payment_method**

```
Заполнение таблицы payment_method
import sqlite3
try:
    sqlite_connection = sqlite3.connect('Berestenkov.db')
    payment_method = '''INSERT INTO payment_method (id, method) VALUES
                            (1, 'Банковская карта'),
.                           (2, 'Мобильный телефон');
.                     '''
    cursor = sqlite_connection.cursor()
    cursor.execute(payment_method)
    sqlite_connection.commit()
    cursor.close()
except sqlite3.Error as error:
    print("Ошибка при подключении к sqlite", error)
finally:
if (sqlite_connection):
sqlite_connection.close()
print("Соединение с SQLite закрыто")
Соединение с SQLite закрыто
```

Листинг 6. Заполнение таблицы **payment_status**

```
# Заполнение таблицы payment_status
import sqlite3
try:
    sqlite_connection = sqlite3.connect('Berestenkov.db')
    payment_status = '''INSERT INTO payment_status (id, status) VALUES
                           (1, 'Оплачено'),
                           (2, 'Не оплачено');
                       '''
    cursor = sqlite_connection.cursor()
    cursor.execute(payment_status)
    sqlite_connection.commit()
    cursor.close()
except sqlite3.Error as error:
    print("Ошибка при подключении к sqlite", error)
finally:
    if (sqlite_connection):
        sqlite_connection.close()
        print("Соединение с SQLite закрыто")


Соединение с SQLite закрыто
```

Итак, справочные таблицы созданы и заполнены необходимой информацией. Для просмотра результата заполнения справочных таблиц используем функцию, представленную в листингах 7-8, с использованием дополнительных библиотек numpy и pandas. [11-12] Результат выполнения запроса, представленного в листинге 8 представлен в Приложении А.

Листинг 7. Реализация функции запроса на чтение базы данных

```
# Функция реализует запрос на чтение базы данных
def query_sql (db, query, visible = True):
  import sqlite3
  sqlite_connection = sqlite3.connect(db)
  cursor = sqlite_connection.cursor()
  try:
    cursor.execute(query)
    sqlite_connection.commit
    if visible:
      rows = cursor.fetchall()
      names = tuple(map(lambda x: x[0], cursor.description))
      import pandas as pd
      import numpy as np
      aa=np.array(rows)
      dd=pd.DataFrame(aa, columns=names)
    if visible:
      return dd
  finally:
    cursor.close()
    sqlite_connection.close()
```

Листинг 8. SQL-запрос на просмотр содержимого справочных таблиц

```
display(query_sql('Berestenkov.db', 'select * from type_transport;'))
display(query_sql('Berestenkov.db', 'select * from type_place;'))
display(query_sql('Berestenkov.db', 'select * from payment_method;'))
display(query_sql('Berestenkov.db', 'select * from payment_status;'))
```

После создания справочных таблиц целесообразно приступить к заполнению целевых таблиц, которые содержат информацию, на хранение которой и нацелена база данных.

Создадим таблицу **User**, так как она не требует дополнительного подключения справочных таблиц. В результате выполнения кода, представленного в листинге 9, создается незаполненная таблица.

Листинг 9. Запрос на создание таблицы **User**

```
# Создаём таблицу Пользователь
query_sql('Berestenkov.db', ''' CREATE TABLE User
                                (id INTEGER PRIMARY KEY NOT NULL,
                                Surname string NOT NULL,
                                Name string NOT NULL,
                                Patronym string NOT NULL,
                                phone VARCHAR(11) NOT NULL,
                                email string NOT NULL);
          ''', False)
```

После создания необходимой таблицы приступим к написанию экранной формы. В результате выполнения кода, представленного в Приложении Б, создается экранная форма, при заполнении которой, соответствующие данные вносятся в таблицу User.

Заполним целевую таблицу при помощи экранной формы.

	id	Surname	Name	Patronym	phone	email
0	1	Берестеньков	Лев	Валерьевич	89653017077	BerestenkovLev@yandex.ru
1	2	Иванов	Иван	Иванови	89015010474	Ivanovii@gmail.com
2	3	Смирнов	Сергей	Сергеевич	74951234567	Smirnovss@yandex.ru
3	4	Зайцева	Мария	Петровна	89039876543	ZaitsevaMaria@yandex.ru
4	5	Орлов	Дмитрий	Сергеевич	89225551234	OrlovDmitry@yandex.ru
5	6	Кузнецов	Андрей	Владимирович	89001112233	KuznetsovAndrey@yandex.ru
6	7	Васильева	Елена	Михайловна	89653017122	VasilyevaElena@yandex.ru
7	8	Федоров	Павел	Александрович	89653467133	FedorovPavel@yandex.ru
8	9	Новикова	Ольга	Юрьевна	89656297142	NovikovaOlga@yandex.ru
9	10	Григорьев	Сергей	Иванович	89654789149	GrigorievSergey@yandex.ru

Рисунок 21 Результат заполнения таблицы User

Создадим таблицу **Parking_space**, для которой уже требуется подключение справочной таблицы: **type_place**. Для подстановки значений из справочной таблицы используется виджет Dropdown, но вначале создадим данную таблицу. В результате работы программного кода, представленного в листинге 10, создается пустая таблица worker.

Листинг 10. Запрос на создание таблицы **Parking_space**

```
# Создаём таблицу Парковочное место
query_sql('Berestenkov.db', ''' CREATE TABLE Parking_space
                                    (id INTEGER PRIMARY KEY NOT NULL,
                                    Parking_area VARCHAR(80) NOT NULL,
                                       Type_place_id INTEGER NOT NULL REFERENCES
type_place (id)
                                    ON DELETE NO ACTION ON UPDATE NO ACTION,
                                    Parking_cost float NOT NULL);
                                ''', False)
```

После создания необходимой таблицы приступим к написанию экранной формы. В результате выполнения кода, представленного в Приложении В, создается экранная форма, при заполнении которой, соответствующие данные вносятся в таблицу **Parking_space**.

Заполним целевую таблицу при помощи экранной формы.

	id	Parking_area	Type_place_id	Parking_cost
0	1	G	1	600.0
1	2	G	1	600.0
2	3	G	1	600.0
3	4	F	1	400.0
4	5	F	1	400.0
5	6	F	1	400.0
6	7	F	2	50.0
7	8	F	1	400.0
8	9	A	1	300.0
9	10	A	1	300.0

Рисунок 22 Результат заполнения таблицы Parking_space

Создадим таблицу **Transport**, требующую не только подключения справочной таблицы, но и получения данных пользователей, оплативших парковочное место.

Листинг 11. Запрос на создание таблицы **Transport**

```
# Создаём таблицу Транспорт
query_sql('Berestenkov.db', ''' CREATE TABLE Transport
                                (id INTEGER PRIMARY KEY NOT NULL,
                                User_id INTEGER NOT NULL REFERENCES User (id)
                                  ON DELETE NO ACTION ON UPDATE NO ACTION,
                                name string NOT NULL,
                                number_transport VARCHAR(8) NOT NULL,
                               type_transport_id INTEGER NOT NULL REFERENCES type_transport
(id)
                                   ON DELETE NO ACTION ON UPDATE NO ACTION);
                            ''', False)
```

После создания необходимой таблицы приступим к написанию экранной формы. В результате выполнения вышеуказанного кода, представленного в Приложении Г, создается экранная форма, при заполнении которой, соответствующие данные вносятся в таблицу Transport.

Заполним целевую таблицу при помощи экранной формы.

	id	User_id	name	number_transport	type_transport_id
0	1	1	Toyota Camry	A123MP797	1
1	2	2	Can-Am Spyder	К234КР797	2
2	3	3	№ 158	Г378УГ77	4
3	4	5	КамАЗ 65115	О567ОО750	3
4	5	6	УРАЛ 4320	С789ТУ797	3
5	6	7	BMW 5 Series	М890ТУ750	1
6	7	4	Kawasaki Ninja 250	Е321КМ77	2
7	8	8	Volkswagen Passat	В456КР77	1
8	9	9	Hyundai Solaris	С789ТУ77	1
9	10	10	Kia Rio	Е321ГР750	1

Рисунок 23 Результат заполнения таблицы Transport

Создадим целевую таблицу **Parking**, отвечающую за постановку транспортного средства на парковочное место

Листинг 12. Запрос на создание таблицы **Parking**

```
# Создаём таблицу Зона парковки
query_sql('Berestenkov.db', ''' CREATE TABLE Parking
                                    (id INTEGER PRIMARY KEY NOT NULL,
                                  Parking_space_id INTEGER NOT NULL REFERENCES Parking_space
(id)
                                    ON DELETE NO ACTION ON UPDATE NO ACTION,
                                   Transport_id INTEGER NOT NULL REFERENCES Transport (id)
                                    ON DELETE NO ACTION ON UPDATE NO ACTION,
                                    Parking_time STRING NOT NULL,
                                    Parking_date DATE NOT NULL);
                                ''', False)
```

После создания необходимой таблицы приступим к написанию экранной формы. В результате выполнения вышеуказанного кода, представленного в Приложении Д, создается экранная форма, при заполнении которой, соответствующие данные вносятся в таблицу.

Заполним целевую таблицу при помощи экранной формы.

	id	Parking_space_id	Transport_id	Parking_time	Parking_date
0	1	1	4	12:40	2025-01-06
1	2	2	5	16:00	2025-01-06
2	3	3	3	23:00	2025-01-07
3	4	4	1	13:00	2025-01-08
4	5	5	6	15:30	2025-01-08
5	6	6	8	18:00	2025-01-08
6	7	7	9	20:00	2025-01-08
7	8	8	10	23:00	2025-01-08
8	9	9	2	20:30	2025-01-09
9	10	10	7	21:30	2025-01-09

Рисунок 24 Результат заполнения таблицы Parking

Создадим целевую таблицу **Payment**, отвечающую за оплату парковочного места после постановки транспортного средства и

использующую сразу две справочные таблицы: **payment_method**, **payment_status**.

Листинг 13. Запрос на создание таблицы **Payment**

```
# Создаём таблицу Платежи
query_sql('Berestenkov.db', ''' CREATE TABLE Payment
                                    (id INTEGER PRIMARY KEY NOT NULL,
                                    Parking_id INTEGER NOT NULL REFERENCES Parking(id)
                                    ON DELETE NO ACTION ON UPDATE NO ACTION,
                                    Payment_amount float NOT NULL,
                                    Payment_date DATE NOT NULL,
                                           Payment_method_id  INTEGER  NOT  NULL  REFERENCES
payment_method(id)
                                    ON DELETE NO ACTION ON UPDATE NO ACTION,
                                           Payment_status_id  INTEGER  NOT  NULL  REFERENCES
payment_status(id)
                                    ON DELETE NO ACTION ON UPDATE NO ACTION);
                                ''', False)
```

После создания необходимой таблицы приступим к написанию экранной формы. В результате выполнения вышеуказанного кода, представленного в Приложении Е, создается экранная форма, при заполнении которой, соответствующие данные вносятся в таблицу.

Заполним целевую таблицу при помощи экранной формы.

	id	Parking_id	Payment_amount	Payment_date	Payment_method_id	Payment_status_id
0	1	1	600.0	2025-01-06	2	1
1	2	2	600.0	2025-01-06	2	1
2	3	3	600.0	2025-01-07	1	1
3	4	4	400.0	2025-01-08	1	2
4	5	5	400.0	2025-01-08	2	1
5	6	6	400.0	2025-01-08	1	2
6	7	7	50.0	2025-01-08	2	1
7	8	8	400.0	2025-01-08	1	2
8	9	9	300.0	2025-01-09	1	1
9	10	10	300.0	2025-01-09	1	1

Рисунок 25 Результат заполнения таблицы Payment

Таким образом, все таблицы базы данных построены и заполнены.

3.2. Построение типовых запросов на выборку данных

Для демонстрации работы с базой данных построим ряд типовых запросов.

Для реализации **запроса типа LIKE** выведем информацию со всеми мототранспортными средствами. Результат выполнения программного кода, представленного в листинге 14, представлен на рисунке 26.

Листинг 14 Запрос типа LIKE

```
#Запрос типа LIKE
# Запрос на транспортные средства, государственные регистрационные номера которых оканчиваются на 77
query_sql('Berestenkov.db', '''SELECT
                                Transport.id AS 'ID Транспорта',
                                Transport.name AS 'Название транспорта',
                                Transport.number_transport AS 'Гос.номер',
                                type_transport.type AS 'Тип транспорта'
                                FROM Transport
                                JOIN type_transport ON Transport.type_transport_id = type_transport.id
                                WHERE number_transport LIKE '%77';
                            ''')
```

	ID Транспорта	Название транспорта	Гос.номер	Тип транспорта
0	3	№ 158	Г378УГ77	Общественный транспорт
1	7	Kawasaki Ninja 250	Е321КМ77	Мототранспортное средство
2	8	Volkswagen Passat	В456КР77	Легковой автомобиль
3	9	Hyundai Solaris	С789ТУ77	Легковой автомобиль

Рисунок 26 Результат выполнения запроса типа LIKE

Для реализации **запроса типа BETWEEN**, который используется для выборки данных из диапазона с указанием начала и конца этого диапазона, получим список всех платежей, сделанных в определённый диапазон дат: 2025-01-07 - 2025-01-08. Результат выполнения программного кода, представленного в листинге 15, представлен на рисунке 27.

Листинг 15. Запрос типа BETWEEN

```
#Запрос типа BETWEEN
#Запрос для получения всех платежей, сделанных в определённый диапазон дат:
query_sql('Berestenkov.db', '''SELECT * FROM Payment WHERE Payment_date BETWEEN '2025-01-07' AND '2025-01-08';''')
```

	id	Parking_id	Payment_amount	Payment_date	Payment_method_id	Payment_status_id
0	3	3	600.0	2025-01-07	1	1
1	4	4	400.0	2025-01-08	2	1
2	5	5	400.0	2025-01-08	2	1
3	6	6	400.0	2025-01-08	2	1
4	7	7	50.0	2025-01-08	2	1
5	8	8	400.0	2025-01-08	2	1

Рисунок 27 Результат выполнения запроса типа BETWEEN

Для реализации **запроса типа IN**, который позволяет отобрать элементы из заданного списка, получим список всех парковочных мест, которые находятся в зонах парковки F и G. Результат выполнения программного кода, представленного в листинге 16, представлен на рисунке 28.

Листинг 16. Запрос типа IN

```
#Запрос типа IN
#Запрос для получения всех парковочных мест, которые имеют определённые типы:
query_sql('Berestenkov.db', '''SELECT * FROM Parking_space WHERE Parking_area IN ('F', 'G');''')
```

	id	Parking_area	Type_place_id	Parking_cost
0	1	G	1	600.0
1	2	G	1	600.0
2	3	G	1	600.0
3	4	F	1	400.0
4	5	F	1	400.0
5	6	F	1	400.0
6	7	F	2	50.0
7	8	F	1	400.0

Рисунок 28 Результат выполнения запроса типа IN

Для реализации **запроса типа SUM** рассчитаем общую сумму всех платежей за парковочные места. Результат выполнения программного кода, представленного в листинге 17, представлен на рисунке 29.

Листинг 17 Запрос типа SUM

```
#Запросы с использованием агрегирующей функцией SUM
#Запос на получение общей суммы всех платежей:
query_sql('Berestenkov.db', '''SELECT SUM(Payment_amount) AS 'Общая сумма платежей' FROM
Payment;''')
```

	Общая сумма платежей
0	4050.0

Рисунок 29 Результат запроса типа SUM

Для реализации **запроса типа AVG** рассчитаем среднюю сумму платежа за парковочное место. Результат выполнения программного кода, представленного в листинге 18, представлен на рисунке 30.

Листинг 18 Запрос типа AVG

```
#Запросы с использованием агрегирующей функцией AVG
#Запрос для получения средней суммы платежа:
query_sql(‘Berestenkov.db’, ‘’’SELECT AVG(Payment_amount) AS ‘Средняя сумма платежа’ FROM Payment;’’’)
```

	Средняя сумма платежа
0	405.0

Рисунок 30 Результат запроса типа AVG

Для реализации **запроса типа COUNT** рассчитаем общее количество пользователей, зарегистрировавшихся для дальнейшей постановки транспортного средства на парковочное место. Результат выполнения программного кода, представленного в листинге 19, представлен на рисунке 31.

Листинг 19 Запрос типа COUNT

```
#Запросы с использованием агрегирующей функцией COUNT
# Запрос для подсчета количества пользователей:
query_sql('Berestenkov.db', '''SELECT COUNT(*) AS 'Количество пользователей' FROM User;''')
```

	Количество пользователей
0	10

Рисунок 31 Результат запроса типа COUNT

Попробуем использовать **связывание таблиц через INNER JOIN** и выведем подробную информацию обо всех платёжных операциях. Результат выполнения программного кода, представленного в листинге 20, представлен на рисунке 32.

Листинг 20 Запрос связывание таблиц через INNER JOIN

```
# Запрос типа INNER JOIN на информацию о платёжных операциях
query_sql('Berestenkov.db', '''SELECT
                                    Payment.id AS PaymentID,
                                    Payment.Payment_amount AS Amount,
                                    Payment.Payment_date AS "Payment Date",
                                    Parking.Parking_time AS "Parking Time",
                                    Parking.Parking_date AS "Parking Date",
                                    Transport.name AS "Transport Name",
                                    Transport.number_transport AS Number,
                                    User.Surname AS "User Surname",
                                    User.Name AS  "User Name"
                                FROM Payment
                                    INNER JOIN
                                    Parking ON Payment.Parking_id = Parking.id
                                    INNER JOIN
                                    Transport ON Parking.Transport_id = Transport.id
                                    INNER JOIN
                                    User ON Transport.User_id = User.id;
                                ''')
```

	PaymentID	Amount	Payment Date	Parking Time	Parking Date	Transport Name	Number	User Surname	User Name
0	1	600.0	2025-01-06	12:40	2025-01-06	КамАЗ 65115	О567ОО750	Орлов	Дмитрий
1	2	600.0	2025-01-06	16:00	2025-01-06	УРАЛ 4320	С789ТУ77	Кузнецов	Андрей
2	3	600.0	2025-01-07	23:00	2025-01-07	№ 158	Г378УГ77	Смирнов	Сергей
3	4	400.0	2025-01-08	13:00	2025-01-08	Toyota Camry	А123МР77	Берестеньков	Лев
4	5	400.0	2025-01-08	15:30	2025-01-08	BMW 5 Series	М890ТУ77	Васильева	Елена
5	6	400.0	2025-01-08	18:00	2025-01-08	Volkswagen Passat	В456КР77	Федоров	Павел
6	7	50.0	2025-01-08	20:00	2025-01-08	Hyundai Solaris	С789ТУ77	Новикова	Ольга
7	8	400.0	2025-01-08	23:00	2025-01-08	Kia Rio	Е321ГР77	Григорьев	Сергей
8	9	300.0	2025-01-09	20:30	2025-01-09	Can-Am Spyder	К234КР77	Иванов	Иван
9	10	300.0	2025-01-09	21:30	2025-01-09	Kawasaki Ninja 250	Е321КМ77	Зайцева	Мария

Рисунок 32 Результат запроса связывание таблиц через INNER JOIN

Попробуем написать запрос со связыванием таблиц с помощью **WHERE** и наложим дополнительные условия: сформируем список пользователей с легковыми автомобилями

Листинг 21 Запрос со связыванием таблиц с использованием WHERE и наложением дополнительного условия

```
#Запросы со связыванием таблиц с использованием WHERE и наложением дополнительного условия
#Запрос Пользователей с легковых автомобилей
query_sql('Berestenkov.db', '''SELECT
                                  User.id AS 'ID Пользователя',
                                  User.Surname AS 'Фамилия',
                                  User.Name AS 'Имя',
                                  User.Patronym AS 'Отчество',
                                  Transport.name AS 'Название Транспорта',
                                 Transport.number_transport AS 'Гос.номер транспорта',
                                  type_transport.type as 'Тип транспорта'
                                FROM User,
                                  Transport,
                                  type_transport
                                WHERE User.id = Transport.user_id
                                  and Transport.type_transport_id = type_transport.id
                                  and type_transport.type = 'Легковой автомобиль';
                               ''')
```

	ID Пользователя	Фамилия	Имя	Отчество	Название Транспорта	Гос.номер транспорта	Тип транспорта
0	1	Берестеньков	Лев	Валерьевич	Toyota Camry	A123MP77	Легковой автомобиль
1	7	Васильева	Елена	Михайловна	BMW 5 Series	M890TY77	Легковой автомобиль
2	8	Федоров	Павел	Александрович	Volkswagen Passat	B456KP77	Легковой автомобиль
3	9	Новикова	Ольга	Юрьевна	Hyundai Solaris	C789TY77	Легковой автомобиль
4	10	Григорьев	Сергей	Иванович	Kia Rio	E321GP77	Легковой автомобиль

Рисунок 33 Результат запроса со связыванием таблиц с использованием WHERE и наложением дополнительного условия

Исследуем **связывание таблиц через LEFT JOIN** - выведем информацию о всех пользователях с их транспортными средствами

Листинг 22 Запрос связывание таблиц через LEFT JOIN

```
#Запросы со связыванием таблиц LEFT JOIN
#Запрос всех пользователей с их транспортом, если нет привязанных транспортов, то выводится None
query_sql('Berestenkov.db', '''SELECT
                                    User.id AS 'ID Пользователя',
                                    User.Surname AS 'Фамилия',
                                    User.Name AS 'Имя',
                                    User.Patronym AS 'Отчество',
                                    Transport.name AS 'Название Транспорта',
                                   Transport.number_transport AS 'Гос.номер транспорта',
                                    type_transport.type as 'Тип транспорта'
                                  FROM User
                                     LEFT JOIN Transport ON User.id = Transport.user_id
                                  LEFT JOIN type_transport ON Transport.type_transport_id = type_transport.id
                                ''')
```

	ID Пользователя	Фамилия	Имя	Отчество	Название Транспорта	Гос.номер транспорта	Тип транспорта
0	1	Берестеньков	Лев	Валерьевич	Toyota Camry	A123MP77	Легковой автомобиль
1	2	Иванов	Иван	Иванович	Can-Am Spyder	К234КР77	Мототранспортное средство
2	3	Смирнов	Сергей	Сергеевич	№ 158	Г378УГ77	Общественный транспорт
3	4	Зайцева	Мария	Петровна	Kawasaki Ninja 250	Е321КМ77	Мототранспортное средство
4	5	Орлов	Дмитрий	Сергеевич	КамАЗ 65115	О567ОО750	Грузовой транспорт
5	6	Кузнецов	Андрей	Владимирович	УРАЛ 4320	С789ТУ77	Грузовой транспорт
6	7	Васильева	Елена	Михайловна	BMW 5 Series	М890ТУ77	Легковой автомобиль
7	8	Федоров	Павел	Александрович	Volkswagen Passat	В456КР77	Легковой автомобиль
8	9	Новикова	Ольга	Юрьевна	Hyundai Solaris	С789ТУ77	Легковой автомобиль
9	10	Григорьев	Сергей	Иванович	Kia Rio	Е321ГР77	Легковой автомобиль

Рисунок 34 Результат запроса связывание таблиц через LEFT JOIN

Заметим также, что **запросы типа RIGHT JOIN** sqlite не поддерживает в чистом виде. Чтобы выбрать главной правую таблицу, ее просто нужно сделать левой и использовать LEFT JOIN. Попробуем смоделировать запрос на обнаружение некорректной ситуации и вывести список транспорта без привязки к пользователю.

Листинг 23 Запрос связывание таблиц через LEFT JOIN для реализации RIGHT JOIN

```
#Запрос со связыванием таблиц RIGHT JOIN (в sqllite не поддерживается, а
реализовывается при помощи LEFT JOIN)
#Запрос Транспорта без привязки к пользователю
query_sql('Berestenkov.db', '''SELECT
                                    Transport.number_transport AS 'Гос.номер
транспорта',
                                    Transport.name AS 'Название Транспорта',
                                    User.id AS 'ID Пользователя',
                                    User.Surname AS 'Фамилия',
                                    User.Name AS 'Имя',
                                    User.Patronym AS 'Отчество'
                                  FROM Transport
                                     LEFT JOIN User ON User.id =
Transport.user_id
                                ''')
```

	Гос.номер транспорта	Название Транспорта	ID Пользователя	Фамилия	Имя	Отчество
0	A123MP77	Toyota Camry	1	Берестеньков	Лев	Валерьевич
1	K234KP77	Can-Am Spyder	2	Иванов	Иван	Иванович
2	Г378УГ77	№ 158	3	Смирнов	Сергей	Сергеевич
3	O567OO750	КамАЗ 65115	5	Орлов	Дмитрий	Сергеевич
4	C789TY77	УРАЛ 4320	6	Кузнецов	Андрей	Владимирович
5	M890TY77	BMW 5 Series	7	Васильева	Елена	Михайловна
6	E321KM77	Kawasaki Ninja 250	4	Зайцева	Мария	Петровна
7	B456KP77	Volkswagen Passat	8	Федоров	Павел	Александрович
8	C789TY77	Hyundai Solaris	9	Новикова	Ольга	Юрьевна
9	E321ГР77	Kia Rio	10	Григорьев	Сергей	Иванович

Рисунок 35 Результат запроса связывание таблиц через LEFT JOIN для реализации RIGHT JOIN

Напишем 2 **запроса с подзапросами** для получения количества оплаченных платежей и для Получение типов транспорта и количества зарегистрированных транспортных средств каждого типа. Результат выполнения программного кода, представленного в листингах 23-24, представлен на рисунках 36-37.

Листинг 23 Запрос с подзапросом для получения количества оплаченных платежей

```
 Запрос с подзапросом для получения количества оплаченных платежей
query_sql('Berestenkov.db', '''SELECT COUNT(*) AS 'Количество оплаченных платежей'
                                FROM Payment
                                WHERE Payment_status_id = (SELECT id FROM payment_status
WHERE status = 'Оплачено');
                                ''')
```

	Количество оплаченных платежей
0	7

Рисунок 36 Результат запроса на получение количества оплаченных платежей

Листинг 24 Запрос с подзапросом для получения пользователей с количеством их транспортных средств

```
query_sql('Berestenkov.db', '''SELECT type_transport.type AS 'Тип
транспорта',
                                (SELECT COUNT(*)
                                 FROM Transport
                                 WHERE Transport.type_transport_id =
type_transport.id) AS 'Количество транспортных средств'
                                FROM type_transport;
                            ''')
```

	Тип транспорта	Количество транспортных средств
0	Легковой автомобиль	5
1	Мототранспортное средство	2
2	Грузовой транспорт	2
3	Общественный транспорт	1

Рисунок 37 Результат запроса на получение типов транспорта и количества зарегистрированных транспортных средств каждого типа

Выводы по 3 главе

Была выбрана система управления базами данных (СУБД) SQLite и разработан код на Python для создания и заполнения всех таблиц, включая

справочные. Продемонстрирована работа с базой данных с помощью различных типов запросов (LIKE, BETWEEN, IN, SUM, AVG, COUNT, INNER JOIN, LEFT JOIN), что подтверждает функциональность и эффективность разработанной физической модели. Глава демонстрирует практическое применение теоретических знаний и навыков программирования для реализации проекта.

ЗАКЛЮЧЕНИЕ

В работе была поставлена цель разработать базу данных для учета предоставления государственной услуги по организации парковочных пространств в городе Москве, для достижения которой необходимо было решить следующие задачи: провести анализ существующих систем учета и управления парковочными пространствами, построить даталогическую и концептуальные модели базы данных, а также разработать физическую модель базы данных, с построением типовых запросов на выборку..

Для достижения цели работы в первой главе был проведен анализ предметной области, который включал изучение текущих проблем с парковкой в Москве, существующих решений и потребностей пользователей. В результате был разработан концептуальный подход к организации базы данных, учитывающий требования как пользователей, так и органов власти.

В работе была произведена нормализация отношений, заключающаяся в структурировании данных и устранении избыточности. Это позволило создать эффективную даталогическую модель, которая включает основные сущности, такие как пользователи, транспортные средства и парковочные места, а также их взаимосвязи.

На основе представленной даталогической модели была разработана физическая модель базы данных, которая обеспечивает функционал для хранения и обработки информации о парковочных местах и пользователях. Реализованная база данных позволяет эффективно управлять данными, проводить анализ использования парковочных пространств и взаимодействовать с пользователями через удобный интерфейс.

На основании вышеизложенного можно считать, что задачи, поставленные в курсовой работе, решены, и цель работы достигнута. Разработанная база данных представляет собой важный инструмент для улучшения управления парковочными пространствами в Москве, что в свою очередь будет способствовать повышению качества жизни горожан и улучшению транспортной ситуации в столице.

СПИСОК ЛИТЕРАТУРЫ

1. Приказ № 543-ПП от 05.10.2012 О проведении пилотного проекта по организации платных городских парковок в городе Москве URL: https://parking.mos.ru/upload/iblock/e7b/k7h3qctw2lpad44j25borq2mmz1zv52k.pdf (дата обращения: 25.12.2024)
2. Приказ № 289-ПП от 17.05.2013 0б организации платных городских парковок в городе Москве URL: https://parking.mos.ru/upload/iblock/e7b/k7h3qctw2lpad44j25borq2mmz1zv52k.pdf (дата обращения: 25.12.2024)
3. Официальный сайт ГКУ «АМПП» URL: https://parking.mos.ru URL: https://parking.mos.ru/upload/iblock/f16/ewv20blhnfej3aci4pqnfwvby2ea1yjg.pdf (дата обращения: 06.01.2025)
4. Приказ № МПП-61-494/24 от 29.11.2024 - О внесении изменений в приказ Государственного казенного учреждения города Москвы "Администратор Московского парковочного пространства" от 27.03.2024 № МПП-06-124/24 URL: https://parking.mos.ru/upload/iblock/836/317y132s6ym3t0a2pbz3fdg27whw32wp.pdf (дата обращения: 07.01.2025)
5. Приказ № 559-ПП от 21.08.2013 Постановление о внесении изменений в правовые акты Правительства Москвы URL: https://parking.mos.ru/upload/iblock/f16/ewv20blhnfej3aci4pqnfwvby2ea1yjg.pdf (дата обращения: 06.01.2025)
6. Проект платных парковок URL: https://parking.mos.ru/extra/project/ (дата обращения: 07.01.2025)
7. Оплата парковки URL: https://parking.mos.ru/parking/street/payments/ (дата обращения: 08.01.2025)
8. Базы данных // Oracle.com URL: https://www.oracle.com/cis/database/what-is-a-relational-database/ (дата обращения: 03.01.2025)

9. About SQLite // SQLite.org URL: https://www.sqlite.org/about.html (дата обращения: 07.01.2025).
10. Well-Known Users of SQLite // SQLite.org URL: https://www.sqlite.org/famous.html (дата обращения: 07.01.2025).
11. NumPy documentation // NumPy URL: https://numpy.org/doc/stable/ (дата обращения: 04.01.2025).
12. Pandas documentation // pandas URL: https://pandas.pydata.org/docs/ (дата обращения: 02.01.2024)
13. Родионов А. Н. Критерии качества даталогических схем. Полнота моделей данных // Вестник Хабаровского государственного университета экономики и права. 2011. №2. URL: https://cyberleninka.ru/article/n/kriterii-kachestva-datalogicheskih-shem-polnota-modeley-dannyh (дата обращения: 08.01.2025)
14. Смирнов М.В., Толмасов Р.С. ГРАФИЧЕСКАЯ НОТАЦИЯ МОДЕЛИРОВАНИЯ ДОКУМЕНТНЫХ БАЗ ДАННЫХ // Открытое образование. 2021. №5. URL: https://cyberleninka.ru/article/n/graficheskaya-notatsiya-modelirovaniya-dokumentnyh-baz-dannyh (дата обращения: 08.01.2025)
15. Widget List // Ipywidgets. URL: https://ipywidgets.readthedocs.io/en/latest/examples/Widget%20List.html (дата обращения: 08.01.2025)
16. Парковка в Москве. URL: https://ru.wikipedia.org/wiki/Парковка_в_Москве (дата обращения: 08.01.2025)
17. Согласие на обработку персональных данных посетителей веб-сайта (https://parking.mos.ru/) и мобильного приложения «Парковки России». URL https://parking.mos.ru/policy/agreement.pdf (дата обращения: 08.01.2025)

18. Абонементные парковки. URL https://parking.mos.ru/parking/barrier/subscribe/ (дата обращения: 08.01.2025)
19. Тариф на платных парковках у поликлиник. URL https://parking.mos.ru/parking/street/tarif-na-platnykh-parkovkakh-u-poliklinik/ (дата обращения: 08.01.2025)
20. Территория платных парковок URL https://parking.mos.ru/parking/street/paid-parking-area/ (дата обращения: 08.01.2025)
21. Места для владельцев парковочных разрешений. URL https://parking.mos.ru/parking/street/special/ (дата обращения: 08.01.2025)

ПРИЛОЖЕНИЕ А

Содержание справочных таблиц

	id	type
0	1	Легковой автомобиль
1	2	Мототранспортное средство
2	3	Грузовой транспорт
3	4	Общественный транспорт

Рисунок 1 Содержание справочной таблицы **type_trasport**

	id	type
0	1	Обычное
1	2	Инвалидное

Рисунок 2 Содержание справочной таблицы **type_place**

	id	method
0	1	Банковская карта
1	2	Мобильный телефон

Рисунок 3 Содержание справочной таблицы **payment_method**

	id	status
0	1	Оплачено
1	2	Не оплачено

Рисунок 4 Содержание справочной таблицы **payment_status**

Рисунок 5

ПРИЛОЖЕНИЕ Б

Создание экранной формы для таблицы User

Листинг 1. Программный код

```
# Виджет таблицы User
import ipywidgets as widgets  # Подключаем библиотеку виджетов
from IPython.display import display  # Импортируем функцию для визуализации виджетов

style = {'description_width': 'initial'}  # Устанавливаем ширину поля description под размер текста в нем

# Функция для вставки данных в таблицу User
def create_user_widgets():
    # Используем Label для отображения заголовка
    header = widgets.Label(value='Заполнение таблицы User')
    display(header)

       user_id = widgets.IntText(value=1, description='ID пользователя:', disabled=False)
     surname = widgets.Text(value='', description='Фамилия:', disabled=False, style=style)
       name = widgets.Text(value='', description='Имя:', disabled=False, style=style)
    patronym = widgets.Text(value='', description='Отчество:', disabled=False, style=style)
      phone = widgets.Text(value='', description='Телефон:', disabled=False, style=style)
      email = widgets.Text(value='', description='Email:', disabled=False, style=style)
    button = widgets.Button(description='Записать в БД', disabled=False)

    # Отображаем виджеты
    display(user_id, surname, name, patronym, phone, email, button)

    def button_clicked(b):
        query = 'INSERT INTO User (id, Surname, Name, Patronym, phone, email) VALUES '
        query += '(' + str(user_id.value) + ', ' + '\'' + surname.value + '\'' + ', ' + \
                 '\'' + name.value + '\'' + ', ' + '\'' + patronym.value + '\'' + ', ' + \
                  '\'' + phone.value + '\'' + ', ' + '\'' + email.value + '\'' + ');'
        print(query)  # Выводим SQL-запрос для проверки
        query_sql('Berestenkov.db', query, False)  # Вставляем данные в БД

    button.on_click(button_clicked)

# Вызов функции для отображения виджетов
create_user_widgets()
```

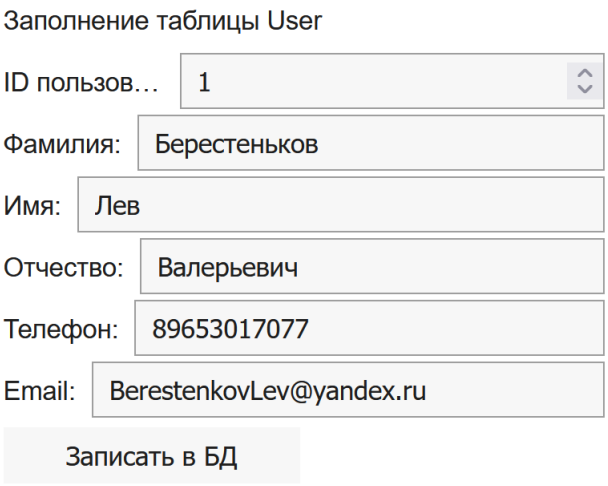


Рисунок 1 Экранная форма таблицы User

ПРИЛОЖЕНИЕ В

Создание экранной формы для таблицы Parking_space

Листинг 1. Программный код

```
# Виджет таблицы Parking_space
import ipywidgets as widgets  # Подключаем библиотеку виджетов
from IPython.display import display  # Импортируем функцию для визуализации виджетов
import sqlite3  # Импортируем библиотеку для работы с SQLite

# Функция для получения типов парковочных мест из справочной таблицы
def get_type_place_options():
    connection = sqlite3.connect('Berestenkov.db')
    cursor = connection.cursor()

    cursor.execute("SELECT id, type FROM type_place;")
    options = cursor.fetchall()  # Получаем все результаты
    cursor.close()
    connection.close()

    return options

# Функция для вставки данных в таблицу Parking_space
def create_parking_space_widgets():
    header = widgets.Label(value='Заполнение таблицы Parking_space')
    display(header)

    # Создаем виджет для ввода ID парковочного места
        parking_space_id   =   widgets.IntText(value=1,   description='ID   парковочного   места:',
disabled=False)

    parking_area = widgets.Text(value='', description='Парковочная зона:', disabled=False)

    # Получаем доступные типы парковочных мест из справочной таблицы
    type_place_options = get_type_place_options()

    # Создаем выпадающий список для выбора типа парковочного места
    type_place_dropdown = widgets.Dropdown(
        options=[(type_name, type_id) for type_id, type_name in type_place_options],
        description='Тип парковочного места:',
        disabled=False
    )

    parking_cost = widgets.FloatText(value=0.0, description='Стоимость парковки:', disabled=False)
    button = widgets.Button(description='Записать в БД', disabled=False)

    # Отображаем все виджеты
    display(parking_space_id, parking_area, type_place_dropdown, parking_cost, button)

    def button_clicked(b):
        query = "INSERT INTO Parking_space (id, Parking_area, Type_place_id, Parking_cost) VALUES "
        query += '(' + str(parking_space_id.value) + ', ' + \
                 '\'' + parking_area.value + '\'' + ', ' + \
                 str(type_place_dropdown.value) + ', ' + \
                 str(parking_cost.value) + ');'
        print(query)  # Выводим SQL-запрос для проверки
        query_sql('Berestenkov.db', query, False)  # Вставляем данные в БД

    button.on_click(button_clicked)



# Вызов функции для отображения виджетов
create_parking_space_widgets()
```

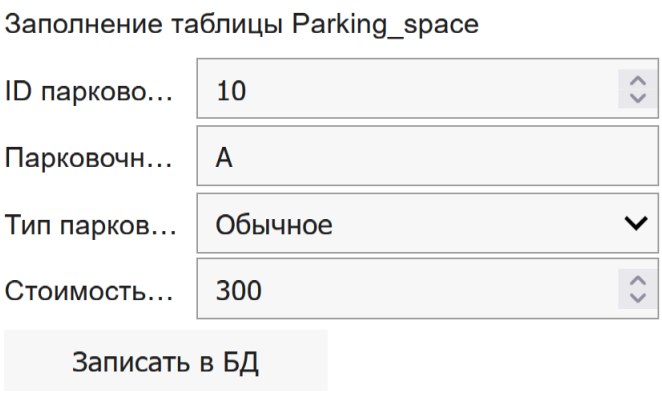


Рисунок 1 Экранная форма таблицы Parking_space

ПРИЛОЖЕНИЕ Г

Создание экранной формы для таблицы Transport

Листинг 1. Программный код

```
# Виджет таблицы Transport
import ipywidgets as widgets  # Подключаем библиотеку виджетов
from IPython.display import display  # Импортируем функцию для визуализации виджетов
import sqlite3  # Импортируем библиотеку для работы с SQLite
import pandas as pd

# Функция для получения типов автомобилей из справочной таблицы
def get_type_car_options():
    try:
        R = query_sql('Berestenkov.db', 'SELECT id, type FROM type_transport;')
         return R if R is not None else pd.DataFrame()  # Возвращаем пустой DataFrame при ошибке
    except Exception as e:
        print("Ошибка при получении типов автомобилей:")
        return pd.DataFrame()  # Возвращаем пустой DataFrame при ошибке

# Функция для получения пользователей из таблицы User
def get_user_options():
    try:
        R = query_sql('Berestenkov.db', 'SELECT id, Surname FROM User;')
         return R if R is not None else pd.DataFrame()  # Возвращаем пустой DataFrame при ошибке
    except Exception as e:
        print("Ошибка при получении пользователей:",)
        return pd.DataFrame()  # Возвращаем пустой DataFrame при ошибке

# Функция для вставки данных в таблицу Transport
def create_transport_widgets():
    # Используем Label для отображения заголовка
    header = widgets.Label(value='Заполнение таблицы Transport')
    display(header)

    # ID транспорта
      transport_id  =  widgets.IntText(value=1,  description='ID  транспорта:', disabled=False)

    # Получаем пользователей из базы данных
    user_options = get_user_options()
    if user_options.empty:
        print("Нет доступных пользователей.")
        return  # Завершаем выполнение функции, если нет пользователей

```

```
    # Создаем выпадающий список для выбора пользователя
    user_dropdown = widgets.Dropdown(
                                        options=[(user_options.iloc[i]['Surname'], int(user_options.iloc[i]['id'])) for i in range(len(user_options))],
        value=int(user_options.iloc[0]['id']),  # Значение по умолчанию (первый пользователь)
        description='Пользователь:',
        disabled=False
    )

    # Название транспорта
       name   =   widgets.Text(value='',   description='Название   транспорта:', disabled=False)

    # Государственный номер транспорта
    number_transport = widgets.Text(value='', description='Государственный номер транспорта:', disabled=False)

    # Получаем типы автомобилей из базы данных
    R = get_type_car_options()

    # Создаем выпадающий список для выбора типа автомобиля
    type_car_dropdown = widgets.Dropdown(
              options=[(R.iloc[i]['type'],  int(R.iloc[i]['id']))  for  i  in range(len(R))],
        value=1,  # Значение по умолчанию
        description='Тип автомобиля:',
        disabled=False
    )

    button = widgets.Button(description='Записать в БД', disabled=False)

    # Отображаем виджеты
         display(transport_id,    user_dropdown,    name,    number_transport, type_car_dropdown, button)

    def button_clicked(b):
         query = "INSERT INTO Transport (id, User_id, name, number_transport, type_transport_id) VALUES "
        query += '(' + str(transport_id.value) + ', ' + \
                 str(user_dropdown.value) + ', ' + \
                 '\'' + name.value + '\'' + ', ' + \
                 '\'' + number_transport.value + '\'' + ', ' + \
                 str(type_car_dropdown.value) + ');'
        print(query)  # Выводим SQL-запрос для проверки
        query_sql('Berestenkov.db', query, False)  # Вставляем данные в БД

    button.on_click(button_clicked)

```

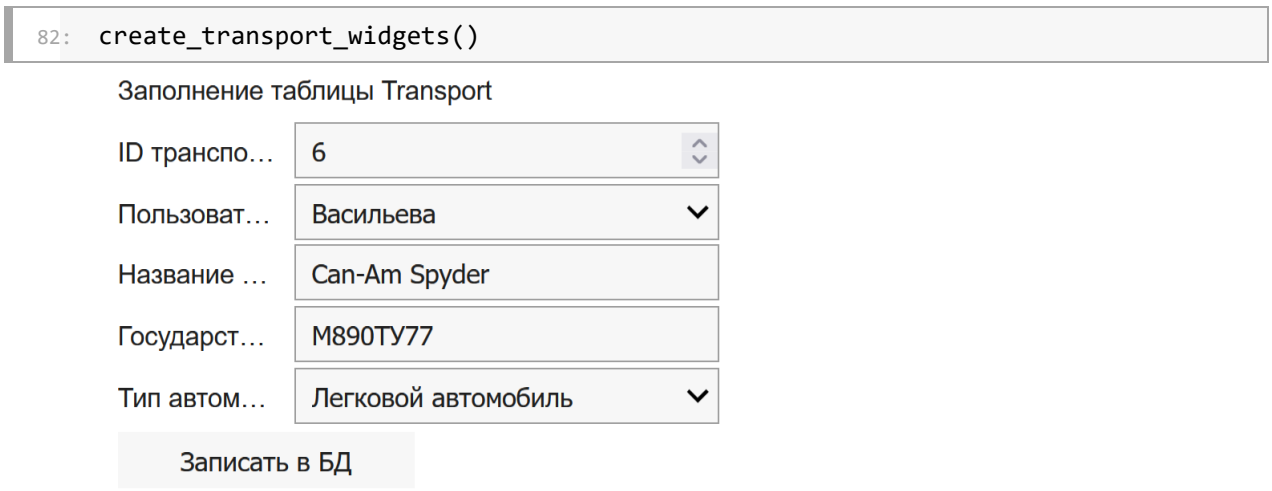


Рисунок 1 Экранная форма таблицы Transport

ПРИЛОЖЕНИЕ Д

Создание экранной формы для таблицы Parking

```
# Виджет таблицы Parking
import ipywidgets as widgets  # Подключаем библиотеку виджетов
from IPython.display import display  # Импортируем функцию для визуализации виджетов
import sqlite3  # Импортируем библиотеку для работы с SQLite

# Функция для получения парковочных мест из таблицы Parking_space
def get_parking_space_options():
    connection = sqlite3.connect('Berestenkov.db')
    cursor = connection.cursor()

    cursor.execute("SELECT id, Parking_area FROM Parking_space;")
    options = cursor.fetchall()  # Получаем все результаты
    cursor.close()
    connection.close()

    return options

# Функция для получения транспортных средств из таблицы Transport
def get_transport_options():
    connection = sqlite3.connect('Berestenkov.db')
    cursor = connection.cursor()

    cursor.execute("SELECT id, name FROM Transport;")
    options = cursor.fetchall()  # Получаем все результаты
    cursor.close()
    connection.close()

    return options

# Функция для вставки данных в таблицу Parking
def create_parking_widgets():
    header = widgets.Label(value='Заполнение таблицы Parking')
    display(header)

    # Создаем виджет для ввода ID парковки
    parking_id = widgets.IntText(value=1, description='ID парковки:', disabled=False)

    # Получаем доступные парковочные места и транспортные средства
    parking_space_options = get_parking_space_options()
    transport_options = get_transport_options()

    # Создаем выпадающий список для выбора парковочного места
    parking_space_dropdown = widgets.Dropdown(
            options=[(f"{area} (ID: {space_id})", space_id) for space_id, area in
parking_space_options],
        description='Парковочное место:',
        disabled=False
    )

    # Создаем выпадающий список для выбора транспорта
    transport_dropdown = widgets.Dropdown(
         options=[(f"{name} (ID: {transport_id})", transport_id) for transport_id, name
in transport_options],
        description='Транспорт:',
        disabled=False
    )

    parking_time = widgets.Text(value='', description='Время парковки (HH:MM):')

    parking_date = widgets.DatePicker(description='Дата парковки:')

```

```
    button = widgets.Button(description='Записать в БД', disabled=False)

    # Отображаем все виджеты
       display(parking_id,  parking_space_dropdown,  transport_dropdown,  parking_time,
parking_date, button)

    def button_clicked(b):
        query = "INSERT INTO Parking (id, Parking_space_id, Transport_id, Parking_time,
Parking_date) VALUES "
        query += '(' + str(parking_id.value) + ', ' + \
                 str(parking_space_dropdown.value) + ', ' + \
                 str(transport_dropdown.value) + ', ' + \
                 '\'' + parking_time.value + '\'' + ', ' + \
                 '\'' + str(parking_date.value) + '\'' + ');'
        print(query)  # Выводим SQL-запрос для проверки

        # Выполняем запрос на вставку данных в БД
        try:
            query_sql('Berestenkov.db', query, False)  # Вставляем данные в БД
            print("Данные успешно записаны в базу данных.")
        except Exception as e:
            print("Ошибка при записи в базу данных:", e)

    button.on_click(button_clicked)

# Вызов функции для отображения виджетов
create_parking_widgets()
```

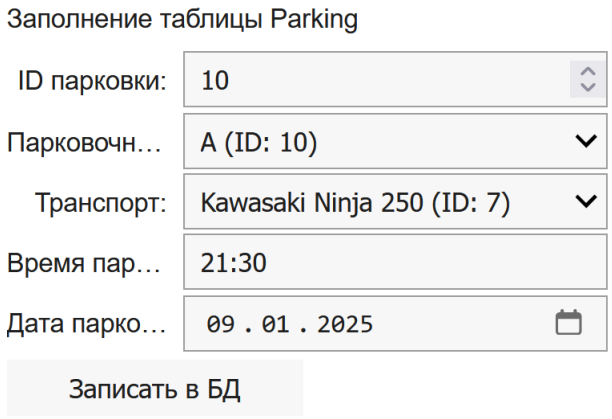


Рисунок 1 Экранная форма таблицы Parking

ПРИЛОЖЕНИЕ Е

Создание экранной формы для таблицы Payment

```
# Виджет таблицы Payment
import ipywidgets as widgets  # Подключаем библиотеку виджетов
from IPython.display import display  # Импортируем функцию для визуализации виджетов
import sqlite3  # Импортируем библиотеку для работы с SQLite

# Функция для получения парковок из таблицы Parking
def get_parking_options():
    connection = sqlite3.connect('Berestenkov.db')
    cursor = connection.cursor()

    cursor.execute("SELECT id, Parking_space_id FROM Parking;")
    options = cursor.fetchall()  # Получаем все результаты
    cursor.close()
    connection.close()

    return options

# Функция для получения методов оплаты из таблицы payment_method
def get_payment_method_options():
    connection = sqlite3.connect('Berestenkov.db')
    cursor = connection.cursor()

    cursor.execute("SELECT id, method FROM payment_method;")
    options = cursor.fetchall()  # Получаем все результаты
    cursor.close()
    connection.close()

    return options

# Функция для получения статусов оплаты из таблицы payment_status
def get_payment_status_options():
    connection = sqlite3.connect('Berestenkov.db')
    cursor = connection.cursor()

    cursor.execute("SELECT id, status FROM payment_status;")
    options = cursor.fetchall()  # Получаем все результаты
    cursor.close()
    connection.close()

    return options

# Функция для вставки данных в таблицу Payment
def create_payment_widgets():
    header = widgets.Label(value='Заполнение таблицы Payment')
    display(header)

    # Создаем виджет для ввода ID платежа
    payment_id = widgets.IntText(value=1, description='ID платежа:', disabled=False)

    # Получаем доступные парковки, методы и статусы оплаты
    parking_options = get_parking_options()
    payment_method_options = get_payment_method_options()
    payment_status_options = get_payment_status_options()

    # Создаем выпадающий список для выбора парковки
    parking_dropdown = widgets.Dropdown(
        options=[(f"Парковка ID: {parking_id} (ID: {parking_space_id})", parking_id) for parking_id, parking_space_id in parking_options],
        description='Парковка:',
        disabled=False
    )
```

```
61:
62:     # Создаем выпадающий список для выбора метода оплаты
63:     payment_method_dropdown = widgets.Dropdown(
64:         options=[(method, method_id) for method_id, method in payment_method_options],
65:         description='Метод оплаты:',
66:         disabled=False
67:     )
68:
69:     # Создаем выпадающий список для выбора статуса оплаты
70:     payment_status_dropdown = widgets.Dropdown(
71:         options=[(status, status_id) for status_id, status in payment_status_options],
72:         description='Статус оплаты:',
73:         disabled=False
74:     )
75:
76:        payment_amount   =   widgets.FloatText(value=0.0,   description='Сумма   платежа:',
    disabled=False)
77:     payment_date = widgets.DatePicker(description='Дата платежа:')
78:
79:     button = widgets.Button(description='Записать в БД', disabled=False)
80:
81:     # Отображаем все виджеты
82:     display(payment_id, parking_dropdown, payment_amount, payment_date,
83:             payment_method_dropdown, payment_status_dropdown, button)
84:
85:     def button_clicked(b):
86:          query = "INSERT INTO Payment (id, Parking_id, Payment_amount, Payment_date,
    Payment_method_id, Payment_status_id) VALUES "
87:         query += '(' + str(payment_id.value) + ', ' + \
88:                  str(parking_dropdown.value) + ', ' + \
89:                  str(payment_amount.value) + ', ' + \
90:                  '\'' + str(payment_date.value) + '\'' + ', ' + \
91:                  str(payment_method_dropdown.value) + ', ' + \
92:                  str(payment_status_dropdown.value) + ');'
93:         print(query)  # Выводим SQL-запрос для проверки
94:
95:         # Выполняем запрос на вставку данных в БД
96:         try:
97:             query_sql('Berestenkov.db', query, False)  # Вставляем данные в БД
98:             print("Данные успешно записаны в базу данных.")
99:         except Exception as e:
100:            print("Ошибка при записи в базу данных:", e)
101:
102:    button.on_click(button_clicked)
103:
104: # Вызов функции для отображения виджетов
105: create_payment_widgets()
```

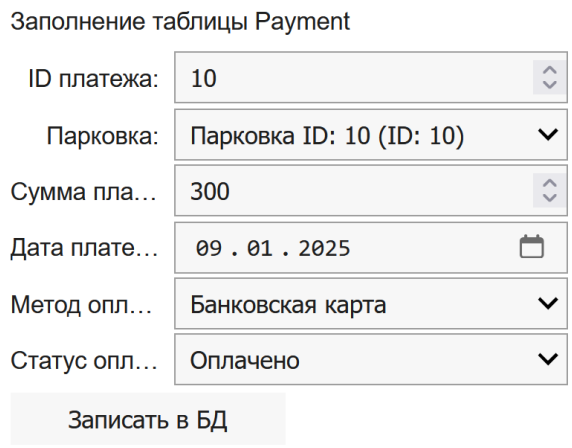


Рисунок 1 Экранная форма таблицы Payment